

UNIT STUDY NOTES

Advanced Python: OOP, Files, Errors and Decorators

The patterns that turn scripts into maintainable software: object-oriented design, file handling, structured error handling, and the decorator and generator mechanisms that underpin most Python frameworks.

 7 source notebooks 231 topic sections extracted 8 study sections

Unit Overview

Topic: 04.01 · **Level:** Intermediate · **Type:** THEORY + CODE

Welcome to Object-Oriented Programming

So far your Python programs have lived inside modules and functions. You have used `len()`, `append()` and `upper()`, but you have never built a *type of your own*. This session changes that. Object-Oriented Programming (OOP) is the idea that data and the operations on that data travel together as one unit — an **object**. Python is an object-oriented language: every value you touch, from a simple integer to a complicated list, is an object that belongs to some class. In this session you will learn to design your own classes, stamp out your own objects, and control how Python's operators behave on them.

What you will learn

1. Function vs Method — which one to use when
2. Creating objects — using a class
3. Defining a class + its constructor
4. `self` and instance attributes
5. Magic methods (`__init__`, `__str__`, `__add__`) — operator overloading

The mental model

Think of a **class** as a blueprint or cookie-cutter and an **object** as the cookie made from it. The blueprint declares which attributes every cookie should have (`flavor`, `size`) and which powers (methods) they all get. When you call the class, a fresh cookie is baked with its own private copy of each attribute. Two objects from the same class look alike on the surface but live in different memory — each carries its own state.

Key definitions

- **Class:** a template/blueprint that bundles data (attributes) and behavior (methods).
- **Object:** a concrete, live instance built from a class.
- **Constructor:** the `__init__` method that automatically runs when an object is born.
- **Method:** a function defined inside a class and called with `object.method()`.

- **Attribute:** a piece of data owned by an object, written `object.attribute`.
- **Magic method:** a dunder (`__...__`) method Python calls behind the scenes for operators and built-ins.

In the cells ahead you will first feel the difference between functions and methods, then build a class, study `self`, and finish with a tiny `Fraction` class where `+`, `-`, `*`, `/` do real fraction arithmetic. By the end you will be able to create reusable, self-contained types just like Python's built-in `list` and `str`. Before you start, remember the shorthand: a **function** takes the object as an argument (`len(L)`), while a **method** is called with a dot inside the type (`L.append()`).

What is OOP

Object-Oriented Programming models entities as objects with state (attributes) and behavior (methods).

Why

Encapsulation, reusability, abstraction, maintainability.

Core ideas

Class (blueprint), Object (instance), Encapsulation, Inheritance (later), Polymorphism (later).

Topic: 04.02 · **Level:** Intermediate · **Type:** THEORY + CODE

Welcome back to OOP

In Session 7 you learned to define classes and create objects. Now you will make objects *interact* and *protect themselves*. A real program is a community of objects: a `Point` talks to another `Point`, a `Person` greets you, an `Atm` guards its balance, and a bowl of scoops tracks a total price. This session gives you the tools those communities rely on.

What you will learn

1. Accessing attributes and methods of objects
2. Reference variables & pass-by-reference
3. Encapsulation — private attributes
4. Collection of objects
5. Static variables & static methods

The scenario

By the end you will be able to write an OOP solution to this concrete problem set:

- A user can create and view 2D coordinates.
- A user can find the distance between two coordinates.
- A user can find the distance of a coordinate from the origin.
- A user can check whether a point lies on a given line.

- A user can find the distance between a 2D point and a given line.

These map beautifully onto two classes: `Point` (its own coordinates plus distance logic) and `Line` (a straight line $Ax + By + C = 0$ plus point-line checks). That design keeps the math inside the object that owns the data.

Key concepts preview

- **Reference variables:** an object lives in memory; a variable only *points* at it, and many variables can point at the same object.
- **Pass by reference:** when you hand an object to a function, the function mutates the very same object — changes are visible outside.
- **Encapsulation:** hiding an object's data behind methods with the `_` prefix so outside code cannot touch it directly.
- **Static members:** class-level variables and `@staticmethod` methods shared by all instances, usable even without an object.

Objects then become not just data holders but *responsible citizens* that expose a clean public interface and keep their own private state consistent. Let's begin with the two geometry classes.

Topic: 04.03 · **Level:** Intermediate · **Type:** THEORY + CODE

Welcome to the third OOP pillar

You already know how to define classes, protect state with encapsulation, and share values with static members. Now you will learn the two relationships that let classes build on each other and behave in many forms: **inheritance** and **polymorphism**.

What you will learn

1. Class relationships — Aggregation (Has-A) and Inheritance (Is-A)
2. Inheritance — what is inherited and what is not
3. The `super` keyword
4. Types of inheritance — single, multilevel, hierarchical, multiple
5. Polymorphism — overriding, overloading, operator overloading
6. Abstraction — ABC (Abstract Base Classes)

The big idea

Real-world concepts relate to each other. An *Order contains items* — a **Has-A** relationship, called aggregation. A *Student is a User* — an **Is-A** relationship, called inheritance. When one class inherits from another, the child automatically receives the parent's non-private attributes and methods: massive code reuse. And when the child redefines a parent method, the same method name behaves differently depending on the class — that is polymorphism, meaning *many forms*.

Map of the session

- **Aggregation:** embed an object inside another (`Customer` has an `Address`).
- **Inheritance basics:** `class Child(Parent):` , what gets inherited, and private members that stay private.
- **Overriding:** the child replaces a parent method with its own version.
- `super()` : call the parent's constructor or methods from inside the child.
- **Inheritance types:** single parent chains down to the diamond problem of multiple inheritance, resolved by Python's MRO.
- **Polymorphism:** overriding, overloading (via default arguments) and operator overloading — the same `+` means different things on different types.
- **Abstraction:** an `ABC` with `@abstractmethod` that *forces* children to implement certain methods.

By the end you will understand how a small class can grow into a family tree of specialized classes that share code yet behave individually.

Topic: 04.04 · **Level:** Intermediate · **Type:** PROJECT

Welcome to your first OOP project

This session stops teaching syntax in isolation and puts it to work: you will build one complete, working application — **NLPApp** — using everything from the last three sessions. Classes, constructors, private attributes, menus, collections and API calls all come together in a single console program.

What NLPApp does

NLPApp is a menu-driven command-line application that lets a user:

- **Register** a new account (name, email, password).
- **Log in** with an existing account.
- Once logged in, run real Natural Language Processing actions:
 - **NER** — named-entity recognition (extract names, places, languages from text).
 - **Language detection** — identify the language of a paragraph.
 - **Sentiment analysis** — classify text as joy, sadness, anger, fear, love or surprise.
- **Log out** and return to the first menu.

The NLP itself comes from the third-party **nlpcloud** API: we send text to a hosted model and parse the JSON it returns. If the API key or network fails during class, do not panic — the deliverable is understanding the *structure*: how a single class orchestrates a whole user journey.

How it will be built

1. An OOP tour of the `nlpcloud` client and installing packages with `!pip install` .
2. Exploring the raw API: entity recognition, language detection, sentiment.
3. Building the `NLPApp` class with private state and menu methods.
4. Parsing the API response — pulling the highest-scoring label out of `scored_labels` .

5. A sprinkle of the `emoji` module for friendly output.

Why a project matters

An application is different from a script: it keeps **state** across interactions, it hides data behind methods, and its code splits into small responsibilities. That is OOP in production. After this session you can genuinely say you have written a small but real software application.

Prerequisite: `!pip install nlpccloud` — install it before running the client cells.

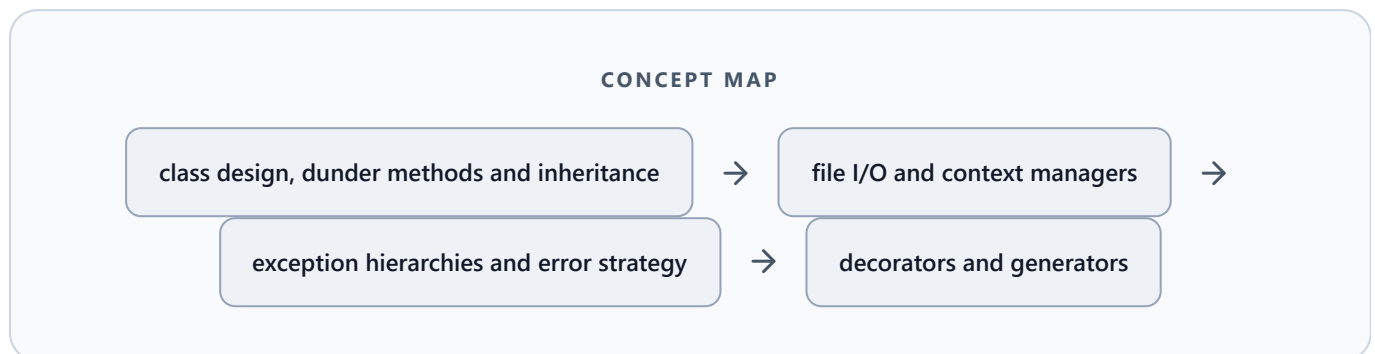
Prerequisites

- Pythonic Thinking
- Functions and scope

What you will be able to do

- ✓ Model a domain with classes, encapsulation, inheritance and polymorphism
- ✓ Read and write files with correct context management and encoding
- ✓ Design exception hierarchies that fail loudly and fail usefully
- ✓ Understand and write decorators and generator-based pipelines

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

OOP Part 1 — Class & Object

1. Function vs Method

1.1 Methods are attached, functions are standalone

Before you write your own classes you need to see the exact difference between a **function** and a **method**, because you will be creating methods all session.

- A **function** lives outside any type. You call it by passing the data in: `len(L)`, `max(L)`, `sorted(L)`.
- A **method** lives inside a type and is reached with a dot: `L.append(4)`, `L.sort()`, `s.upper()`.

The dot is the key clue: `object.method()`. When you write `L.upper()`, Python looks inside the list class, finds no `upper`, and raises `AttributeError: 'list' object has no attribute 'upper'`. Similarly `s.append('x')` fails because strings have no `append`.

CALL STYLE	EXAMPLE	MEANING
Function	<code>len(L)</code>	pass the object in as an argument
Method	<code>L.append(4)</code>	the object calls its own behavior

This is the foundation of **abstraction**: data and the operations that make sense for that data live in the same place. A list knows how to append, sort and pop; a string knows how to upper, split and strip. Each type carries its own little toolbox.

1.2 Deep dive — everything is an object

An object is a bundle of **state** (data) and **behavior** (methods). Every object in Python — an integer, a string, a list, a function — is an instance of some class, and it arrives with the methods of its own type. `type(L)` returns `<class 'list'>`, which tells you which toolbox the object carries.

- Methods are *attached* to an object: `object.method(arguments)`.
- Without attached methods you would have to write a separate function for every data type.
- OOP lets you define your own custom types so the "toolbox" can be exactly what your problem needs.

`L` is a list object: it can `append`, `sort`, and `reverse`, but it has no idea what `upper` means. `s` is a string object: it can `upper` and `split`, but it cannot `append`. The error is the class telling you clearly: "this operation does not exist in my toolbox." You will see both `AttributeError`s when you run the two cells below — that is intentional and instructive.

PYTHON

```
L = [1,2,3]
```

```
L.upper()
```

PYTHON

```
s = 'hello'
s.append('x')
```

2. Object Syntax

2.1 The one-line creation formula

Creating an object in Python is a single line:

Calling a class — for example `list()`, `str()`, or `Person()` — does three things behind the scenes: it reserves space in memory for a new instance, runs the class's constructor so the object gets its initial state, and hands you the finished object. You then keep a label (the variable name) pointing at it.

TERM	MEANING
Class	The blueprint — the design of the thing
Object	The concrete instance built from the blueprint

Every instance gets its own copy of the attributes defined by the class. Two `ClassName()` calls create two separate objects, each with its own data, even though both follow the same blueprint. This is the essence of OOP: **the design is shared, the state is not.**

PYTHON

```
objectname = ClassName()
```

Object syntax

The formula for creating an object is a single line:

What does it do? A **new instance** is allocated in memory that carries its own copy of all the attributes/methods defined by that class.

TERM	MEANING
Class	Blueprint (the design of a thing)
Object	The real instance built from the blueprint

PYTHON

```
objectname = classname()
```

2.2 Deep dive — object: the instance

- **Object** = a concrete instance of a class, a live copy with its own memory.
- **Class** = blueprint/design; **Object** = the actual product.
- Formula: `obj = ClassName()`.
- A list object is `list()` or the literal `[1, 2, 3]`.
- Every object has its own **state** (data) and **behavior** (methods).
- Two objects of the same class differ in data but share behavior.
- In Python literally every value is an object — int, str, list, and so on.
- Creating an object means calling the constructor/ `__init__`.

Run the two cells below: the first just notes the creation syntax in a comment, the second builds `L = [1, 2, 3]`. Both end with an object sitting in memory with a name pointing at it. The next section compares this literal shortcut with the explicit constructor call.

PYTHON

```
# syntax to create an object

#objectname = classname()
```

PYTHON

```
# object literal
L = [1,2,3]
```

3. Constructors & Literals

3.1 Two ways to create a value

Python gives you two routes to the same kind of value — a shorthand called a **literal** and an official **constructor call**.

- `[1, 2, 3]`, `'hello'`, `{1, 2}`, `(1, 2)` are literals — readable and direct.
- `list()`, `str()`, `int()`, `dict()` are constructors — used when you build or convert a value dynamically.
- `str()` gives `''`, `list()` gives `[]`, `int()` gives `0`, `dict()` gives `{}`.

STYLE	EXAMPLE	WHEN TO USE
Literal	<code>[1, 2, 3]</code> , <code>'abc'</code>	readability, fixed values
Constructor	<code>list()</code> , <code>str()</code>	empty values, conversions, data-driven creation

`type(L)` tells you which class an object belongs to — the constructor and the literal produce the same type. The constructor is essentially a tiny factory: call it, and a fresh object rolls off the line.

PYTHON

```
L = [1, 2, 3]    # object Literal (shortcut)
L = list()      # constructor call (zero args)
s = str()       # constructor → ''
```

Constructors & literals

There are 2 ways to create a value:

- `[1, 2, 3]` — literal, direct & readable
- `list()` — constructor, used to **convert a value into a list**
- `type(L)` tells you which class an object belongs to

PYTHON

```
L = [1, 2, 3]    # object literal (shortcut)
L = list()      # constructor call (zero args)
s = str()       # empty string
```

3.2 Deep dive — constructors vs literals

- **Constructor** — `ClassName()` — creates an instance of the class.
- **Literal** — shorthand syntax: `[]`, `()`, `{}`, `' '`, `0`.
- Literals internally still go through the constructor machinery.
- The result is an object of the same type in both cases.
- Literals win on readability; constructors win when creating values dynamically.
- `str()` → `' '`, `list()` → `[]`, `int()` → `0`.
- Think of the constructor as a **factory** — every call produces a new object.

Run the cells below: `L = list()` yields `[]`, and `s = str()` yields `' '`. This matters because every class you write later will have its own constructor call, and `ClassName()` will be the standard way its objects come alive.

PYTHON

```
L = list()
L
```

PYTHON

```
s = str()
s
```

4. Naming Conventions

4.1 Reading code at a glance

Names carry meaning even before you read the code. Python's community follows **PEP 8**, and the most important rule for OOP is: a class name starts with a capital letter.

WHAT	STYLE	EXAMPLE
Class	PascalCase	<code>HelloWorld</code> , <code>Atm</code>
Object/Variable	lowercase	<code>obj1</code> , <code>balance</code>
Function/Method	lowercase + underscore	<code>convert_to_decimal()</code>
Constant	ALL_CAPS	<code>MAX_SPEED</code>

- **PascalCase**: every word capitalized, no spaces — `HelloWorld`.
- **snake_case**: lowercase words joined by underscores — `my_var`, `my_func`.

- Class names are nouns (`Person` , `Car` , `Atm`); methods are verbs (`get_balance` , `withdraw`).

This is coding etiquette: the moment you see `Atm` you know it is a class, and `atm1` tells you it is an instance. Consistency matters more than cleverness — a reader should never have to guess which thing is a type.

Naming conventions

WHAT	STYLE	EXAMPLE
Class	PascalCase	<code>HelloWorld</code> , <code>Atm</code>
Object/Variable	lowercase	<code>obj1</code> , <code>balance</code>
Function/Method	lowercase underscore	<code>convert_to_decimal()</code>

PascalCase = first letter of every word capitalized, without spaces. This is part of coding etiquette — the code reader immediately knows which thing is a class.

4.2 Deep dive — naming conventions

WHAT	STYLE	EXAMPLE
Class	PascalCase	<code>HelloWorld</code>
variable	snake_case	<code>my_var</code>
function/method	snake_case	<code>my_func()</code>
constant	ALL_CAPS	<code>MAX_SPEED</code>

- **PascalCase**: every word capitalized — used for classes, no spaces or underscores.
- **snake_case**: lowercase words with underscores — used for variables, functions, methods.
- camelCase or anything else — consistency is all that matters.
- Class names are nouns (`Person` , `Car` , `Atm`); methods are verbs (`get_balance` , `withdraw`).
- `_private` and `__dunder__` — privacy conventions you meet ahead.
- PEP 8, the official style guide, is what professional projects follow.

The next cell writes the literal `HelloWorld` — the point is only to see that the pattern reads as a class. Good names make the class diagram visible right in the source. `class Person:` is also a statement, executed when the line runs; the class body builds a single blueprint object that every future `Person()` uses as its starting map.

PYTHON
<pre># Pascal Case HelloWorld</pre>

5. Making a Class + Constructor

5.1 The `class` keyword

To create a type of your own you write `class Atm:` and put everything the type owns — attributes and methods — inside its indented body.

Four new ideas appear here:

- `class Atm:` — the blueprint begins. `Atm` is now a user-defined type (PascalCase!).
- `__init__` — the **constructor**: it runs by itself the moment an object is created.
- `self` — a reference to the current object. `self.pin` means "the `pin` this particular object owns", so every object keeps its own copy of `pin` and `balance`.
- `menu()` — a method that can read and change the object's own attributes through `self`.

This is your first full class. The pattern is always `def __init__(self):` for setup, then ordinary methods that take `self` as their first parameter.

PYTHON

```
class Atm:
    def __init__(self):
        self.pin = ''
        self.balance = 0

    def menu(self):
        user_input = input("Hi how can I help you?")
```

Making a class + constructor

- `class Atm:` — the blueprint starts
- `__init__` = **constructor** — called instantly as soon as an object is created
- `self` = reference to the current object — `self.pin` are the attributes that every object keeps its own copy of
- `menu()` — a method that can access its object's attributes

PYTHON

```
class Atm:
    def __init__(self):
        print(id(self))
        self.pin = ''
        self.balance = 0

    def menu(self):
        user_input = input("Hi how can I help you?")
```

5.2 Deep dive — the constructor auto-runs

- Creating an object by calling the class — `Temp()` — makes the constructor run automatically.

- You never write `obj.__init__()` yourself.
- The constructor's job: initialize (set up) the new object's data.
- It is the "on-birth" hook — the first code that runs inside the freshly created object.
- A class can have only one `__init__` (no overloading); give flexibility with default arguments: `def __init__(self, x=5)`.
- Other languages write `new ClassName()`; Python uses the same call syntax as a plain function: `ClassName()`.

Run the `Temp` cell and watch `hello` print with no explicit call — that is the constructor running on its own. Add your own setup logic to `__init__` — setting attributes, printing, opening a file — and it will run at every birth, which is why `__init__` is the natural home for object setup routines.

PYTHON

```
class Temp:
    def __init__(self):
        print('hello')

obj = Temp()    # 'hello' prints - __init__ was auto-called
```

5.3 Deep dive — building a real class with `__init__`

- `class` keyword defines the blueprint.
- `__init__` is the constructor — automatically called when an object is made.
- `self` is the first parameter of every instance method — a reference to the CURRENT object.
- `self.pin = pin` stores an argument into an instance attribute (state).
- A **parameterized constructor** lets you customize each object at birth: `Atm(1234)` creates an ATM with pin `1234`.
- Without a constructor the object would have no initialized data.
- Same idea as Java/C++ constructors — different syntax.

Now look at the big `Atm` class in the upcoming cell: `__init__` sets `pin` and `balance`; `menu`, `create_pin`, `change_pin` and `withdraw` are methods that operate on those attributes through `self`. One honest bug worth spotting in that source: `change_pin()` is missing its `self` parameter, so it cannot reach `self.pin` — a classic slip that the rest of the class helps you recognize.

PYTHON

```

class Atm:

    # constructor(special function)->superpower ->
    def __init__(self):
        print(id(self))
        self.pin = ''
        self.balance = 0
        #self.menu()

    def menu(self):
        user_input = input("""
        Hi how can I help you?
        1. Press 1 to create pin
        2. Press 2 to change pin
        3. Press 3 to check balance
        4. Press 4 to withdraw
        5. Anything else to exit
        """)

        if user_input == '1':
            self.create_pin()
        elif user_input == '2':
            self.change_pin()
        elif user_input == '3':
            self.check_balance()
        elif user_input == '4':
            self.withdraw()
        else:
            # ...
            print('abe garib')
        else:
            print('sale chor')
        self.menu()

```

PYTHON

```

class Atm:
    def __init__(self, pin, balance=1000):
        self.pin = pin
        self.balance = balance

```

6. Creating Objects & Identity

6.1 Two objects, two addresses

Every time you call a class, Python creates a brand-new object:

- `Atm()` allocates fresh memory for a new instance and runs the constructor again.
- `id(object)` returns that object's **unique identity** — effectively its memory address.

- `obj1` and `obj2` come from the same blueprint but live in different places, so each keeps its own `pin` and `balance`.
- Inside the constructor, `print(id(self))` shows the same address as `id(obj1)` — `self` really is the object being created.

Think of `id` as a fingerprint: two objects never share an identity. Checking `id()` is your first debugging tool for the question "is this a new object or the same one?". The upcoming cells show the `Employee` constructor assigning each incoming argument to `self` — watch how every subsequent method call reads the object's own copy of that data, which is the entire value proposition of the instance.

PYTHON

```
obj1 = Atm()
obj2 = Atm()
print(id(obj1))    # unique memory address
print(id(obj2))    # DIFFERENT address
```

Creating objects + identity

- Every `Atm()` call creates a new object — the constructor runs again
- `id(object)` = that object's **unique memory address**
- `obj1` and `obj2` have different ids → different objects, different attributes
- Inside the constructor `print(id(self))` shows the same address that `id(obj1)` gives

PYTHON

```
obj1 = Atm()
obj2 = Atm()
print(id(obj1))
print(id(obj2))
```

6.2 Deep dive — identity, not just equality

- Every `ClassName()` call creates a new, separate object.
- `id()` gives the unique identity (memory location) of an object.
- `obj1` vs `obj2`: same class, different state, different identity.
- By default `obj1 == obj2` is `False`, because the default equality compares identity.
- A reference variable holds the object's address, not a copy of it.
- Two objects → different internal values as a natural property of being distinct.

Run the cells below: you will see two different ids, and `id(obj1)` will match the `id(self)` printed inside the constructor — proof that `self` refers to the very object being built. Different memory means different state: changing `obj1.balance` never touches `obj2`'s copy. Keep the mental picture of two separate memory boxes labeled `obj1` and `obj2` — the blueprint may be shared, but the data inside is theirs alone. References are the next session's topic, but the moral is already clear: identity is the guarantee that two objects do not bleed into each other.

PYTHON

```
obj1 = Atm()
```

PYTHON

```
id(obj1)
```

7. Function vs Method — `len` vs `append`

7.1 Two call styles, two designs

`len` and `append` do different jobs and are called differently:

- `len(L)` — a function living OUTSIDE the list class; you hand it the list.
- `L.append()` — a method living INSIDE the list class; you ask the object itself.
- Similarly `str()`, `list()`, `sorted()` are functions; `s.upper()`, `L.sort()`, `L.pop()` are methods.

OPERATION	STYLE	LIVES
<code>len(L)</code>	function	outside the type
<code>L.append()</code>	method	inside the type

Generic, shared utilities become functions; type-specific behavior becomes methods. Internally, `len(L)` actually delegates to the magic method `L.__len__()` — a preview of the power you gain in the magic-methods section.

PYTHON

```
L = [1, 2, 3]
len(L)      # function - pass the object in as an ARGUMENT
L.append(42) # method - the object calls it with a dot
```

len vs append

- `len(L)` — it is a function, outside the class; pass the list into it
- `L.append()` — it is a method, inside the class; called directly with `.`
- Similarly `str()`, `list()` — functions; `s.upper()`, `L.sort()` — methods

PYTHON

```
len(L)      # function -> you pass the list object as an ARGUMENT
L.append()   # method   -> the object itself calls the method
```

7.2 Deep dive — generic function vs type-specific method

- **Function** (`len`, `min`, `max`, `sorted`) is defined OUTSIDE the object and accepts it as an argument.
- **Method** (`append`, `upper`, `sort`) is defined INSIDE the class and is attached to the object.

- `len` works on strings, lists, tuples — a generic utility.
- `append` works only on lists — type-specific behavior.
- Call style: `func(obj)` versus `obj.method()`.
- Internally, `len` calls the magic method `L.__len__()`.
- Design rule: shared generic operations belong to functions, type-unique ones belong to methods.

Run the cell below — it computes the length of a list *and* calls `append` — and then the constructor-automation section shows how `__init__` runs without being asked. Remember: when Python meets `L.append(4)` it resolves `append` through the list class, but `len(L)` is looked up in the built-in scope — a subtle difference in where the operation lives, and a preview of how `self` will anchor your own methods later.

PYTHON

```
L = [1,2,3]
len(L) # function -> bcos it is outside the list class
L.append()# method -> bcos it is inside the list class
```

PYTHON

```
L = [1, 2, 3]
len(L) # function - works on any sequence
L.append(42) # method - belongs to the list class only
```

8. The Constructor Runs By Itself

8.1 The "on-birth" hook

- `__init__` is never called manually.
- The moment you write `Temp()`, the constructor runs automatically.
- This is why setting up state inside `__init__` is safe — you cannot forget it, it always runs at birth.
- It is the guaranteed first act of every new object's life, seen everywhere in factories and mock objects.

The `Temp` cell prints `hello` with no explicit call — the constructor did its job on its own. The tiny arithmetic cell after it (`3/4*1/2`) is just a warm-up before you meet magic methods, which let you redefine exactly how operators like `+` and `*` behave for your own classes. Keep this mental hook in mind: the constructor establishes the object's data, and the magic methods in the next section let that object speak Python's operator language.

PYTHON

```
class Temp:
    def __init__(self):
        print('hello')

obj = Temp() # hello prints!
```

The constructor runs by itself

- `__init__` is never called manually
- As soon as you write `Temp()` the constructor runs automatically
- This pattern is always seen in factories and mock objects

PYTHON

```
class Temp:

    def __init__(self):
        print('hello')

obj = Temp()
```

PYTHON

```
3/4*1/2
```

9. Magic Methods & Operator Overloading

9.1 Making `+` and `print` understand your class

Python reserves a set of "dunder" (double-underscore) methods for special operations. When you use an operator or a built-in, Python quietly translates it into a magic-method call.

MAGIC METHOD	OPERATOR / BUILT-IN
<code>__str__</code>	<code>print()</code> and string conversion
<code>__add__</code>	<code>+</code>
<code>__sub__</code>	<code>-</code>
<code>__mul__</code>	<code>*</code>
<code>__truediv__</code>	<code>/</code>

By defining these on your own class you **overload the operators** — you decide what `+` means for your objects. The `Fraction` class coming up defines `__str__` so `print(fr1)` shows `3/4`, and `__add__` so `fr1 + fr2` performs real fraction addition. Same operators, your own meaning — this is operator overloading, and it is exactly how Python lets a class feel like a built-in type to everyone using it.

PYTHON

```
fr1 + fr2      # → fr1.__add__(fr2)
print(fr1)     # → uses __str__ / __repr__
len(L)         # → L.__len__()
```

Magic methods & operator overloading

- `__init__(self, x, y)` — constructor with arguments
- `__str__` — gives a readable output for `print(fr1)`
- `__add__` / `__sub__` / `__mul__` / `__truediv__` — operator overloading
- `fr1 + fr2` internally calls `fr1.__add__(fr2)`

PYTHON

```
class Fraction:
    def __init__(self, x, y):      # parameterized constructor
        self.num = x
        self.den = y

    def __str__(self):            # what is shown on print
        return '{}/{ {}'.format(self.num, self.den)

    def __add__(self, other):      # define + in our own way
        new_num = self.num * other.den + other.num * self.den
        new_den = self.den * other.den
        return Fraction(new_num, new_den)
```

Deep Dive : Magic methods — operator overloading

- **Magic methods** (dunder `__...__`) — python calls them internally for special operations
- `__add__` → `+` | `__sub__` → `-` | `__mul__` → `*` | `__truediv__` → `/`
- Operator overloading = defining operators on your own class
- `len(L)` internally `L.__len__()` | `str` → `__str__`
- `print(fr1)` → uses `__str__` / `__repr__`
- When you need it: custom math types (Fraction, Vector, Matrix)
- Formula for fraction addition: cross-multiply shortcut
- Magic methods = Python's "speech interface" for operators
- Favorite interview topic: name a few dunder

PYTHON

```

class Fraction:

    # parameterized constructor
    def __init__(self,x,y):
        self.num = x
        self.den = y

    def __str__(self):
        return '{}/{}'.format(self.num,self.den)

    def __add__(self,other):
        new_num = self.num*other.den + other.num*self.den
        new_den = self.den*other.den

        return '{}/{}'.format(new_num,new_den)

    def __sub__(self,other):
        new_num = self.num*other.den - other.num*self.den
        new_den = self.den*other.den

        return '{}/{}'.format(new_num,new_den)

    def __mul__(self,other):
        new_num = self.num*other.num
        new_den = self.den*other.den

        return '{}/{}'.format(new_num,new_den)

    # ...
    return '{}/{}'.format(new_num,new_den)

    def convert_to_decimal(self):
        return self.num/self.den

```

PYTHON

```

fr1 = Fraction(3,4)
fr2 = Fraction(1,2)

```

10. Every Type Has Its Own Operator Set

10.1 The same symbol, different meanings

Operators are not universal — every type decides which ones it supports.

- `int` and `float` use `+` for addition; `list` and `str` use `+` for concatenation; `set` does not define `+` at all, so it raises `TypeError`.
- Defining the magic method enables the operator; omitting it means an error.
- For `Fraction` you defined `__add__`, so `+` worked; plain sets have no `__add__`, so `+` fails.

TYPE	+ BEHAVIOR
<code>int</code> / <code>float</code>	numeric addition
<code>str</code> / <code>list</code>	concatenation
<code>set</code>	not defined → <code>TypeError</code>
<code>Fraction</code> (yours)	your <code>__add__</code> logic

This is polymorphism in action — the same symbol, with the behavior decided by the class behind it.

```
PYTHON
s1 = {1, 2, 3}
s2 = {3, 4, 5}
s1 + s2  # TypeError - sets do not support +
```

Every type has its own operator set

- Not every type supports every operator
- Set does not define `+` → `TypeError`
- For `Fraction` we defined `__add__` so `+` worked
- Rule: define the magic method = operator enabled; don't define it = error

```
PYTHON
s1 = {1, 2, 3}
s2 = {3, 4, 5}
s1 + s2  # TypeError! sets do not support +
```

Deep Dive : Every type has its own operator set

- Every TYPE defines its own operators (via magic methods)
- set: `|` union and `&` intersection work, but `+` is NOT defined
- list/str: `+` concatenation
- int/float: `+` addition
- You define whichever operators you want on your own class
- So the "operator" meaning = type-dependent
- Same symbol, behavior decided by the class (polymorphism)
- Error is always a type mismatch / unsupported operation
- Knowing the supported operators per type = type fluency

PYTHON

```
s1={1,2,3}
s2={3,4,5}

s1 + s2
```

PYTHON

```
print(fr1 - fr2)
```

Summary

In this session you built the foundation of Python OOP. You learned to distinguish a function (`len(L)`) from a method (`L.append()`), and to see every value as an instance of some class. You created objects with the single-line formula `obj = ClassName()`, met constructors and literals, and followed the naming conventions that make classes readable. You then wrote your first real class — `class Atm:` with an `__init__` constructor, instance attributes stored through `self`, and methods that read and change that state. You verified object identity with `id()`, saw that `self` is the object being created, and understood that each `ClassName()` call stamps out a separate, independent object. Finally, you took control of Python's behavior by defining magic methods: `__str__` made `print(fr1)` show `3/4`, and `__add__`, `__sub__`, `__mul__`, `__truediv__` gave your `Fraction` real arithmetic — and you learned that every type brings its own operator set, which is why sets reject `+`.

This session turned simple classes into a communicating, self-protecting system. You accessed object data with attributes (`p.name`) and behavior with methods (`p.greet()`), learned that reading a missing attribute raises `AttributeError`, and discovered that Python happily lets you *add* attributes from outside for any single object. Reference variables taught you that a name is only a pointer: `q = p` shares one object, functions receive objects by reference, and mutating an object inside a function is visible everywhere the object is known. Encapsulation showed how `__private` attributes guarded by getter/setter methods hide state and add validation. You stored objects inside lists and dicts to iterate and look them up, and finally met class-level members: static variables that count every instance and `@staticmethod` utilities callable without an object. Together these tools let you model systems — points, ATMs, lions, collections — where data stays safe and shared knowledge lives in the right place.

Inheritance and polymorphism complete your OOP toolkit. You distinguished **Has-A** (aggregation — a `Customer` owns an `Address`) from **Is-A** (inheritance — `class Student(User)`), and saw how the child automatically inherits non-private attributes and methods while private `__members` stay sealed inside the parent. When a child defines its own `__init__`, it should call `super().__init__()` to keep the parent's setup — the reason constructor chaining appears everywhere. You explored the four family shapes — single, multilevel, hierarchical, multiple — and met the diamond problem, resolved deterministically by Python's MRO. Overriding, overloading (via default arguments) and operator overloading gave you polymorphism: one name, many behaviors. Finally, `ABC` with `@abstractmethod` let you define contracts that force every child to implement crucial methods. You can now build class families that share code at the base and specialize at the leaves — the structure behind most real-world Python libraries.

This project tied the whole OOP course together. You connected to a hosted NLP API with `nlpcloud.Client(model, key)`, discovered the JSON shapes returned by sentiment, language detection and entity recognition, and installed packages from inside the notebook with `!pip install`. Then you built `NLPApp`, a single class holding a private `__database` of users, whose constructor immediately launches a menu-driven loop: register, login, run NER / language detection / sentiment, and log out. You parsed the API's `scored_labels` by extracting scores, sorting with `enumerate` and a lambda, and picking the top label — and learned that an application is really objects holding state and methods orchestrating a user journey. The emoji module added a friendly finish. Whether the API key works live or not, you now know how to structure a real, stateful, interactive Python application with OOP — the blueprint every real project follows.

Over Session 10 you learned to make Python remember. File I/O follows the ritual *open, read/write, close*: `'w'` creates or truncates, `'a'` appends safely, `'r'` reads; `writelines()` writes whole lists but never adds newlines; `read()`, `readline()`, `readlines()` and chunked pointer reads cover every reading style. The `with` context manager closes files automatically, and `seek()` / `tell()` steer the file pointer in both read and write mode. Binary files demand `'rb'` / `'wb'` because text mode cannot decode images. Text files accept only strings, which pushed us toward serialization: JSON (`dump` / `load` / `dumps` / `loads`) stores basic types readably across languages, needs a `default=` converter for custom objects and supports `indent` for pretty output; pickle stores objects as Python-only binary but is dangerously unsafe with untrusted input. Use JSON for APIs and configs, and pickle only for trusted internal state.

Session 11 turned errors from enemies into tools. You distinguished syntax errors (compile-time failures fixed by correcting grammar) from exceptions (runtime failures handled in code). The built-in family — `IndexError`, `KeyError`, `TypeError`, `ValueError`, `NameError`, `AttributeError`, and more — names its cause, so reading tracebacks is your first diagnostic skill. The `try` / `except` block wrapped around risky code catches problems gracefully; stacked `except` clauses handle each failure type when ordered specific → general; `else` runs only on success; and `finally` always runs, making it the home for cleanup like closing files and connections. With `raise` you throw exceptions deliberately — the right response to invalid business rules such as an overdraw — and custom exception classes (subclasses of `Exception`) signal domain problems by name while fitting into the one hierarchy rooted at `BaseException`. Fail loudly, recover gracefully, and keep your programs readable.

This session connected Python's naming machinery to one of its most expressive features. You learned that every name lives in a namespace — a dictionary of identifiers — and that Python finds it by scanning the LEGB chain: Local, Enclosing, Global, Built-in. That one rule explains shadowing, `UnboundLocalError`, the `global` keyword for module-level reassignment, and the `nonlocal` keyword for editing enclosing-function variables inside closures. You then met closures as the bridge to decorators: because functions are first-class, a higher-order function can receive a function, wrap extra behaviour around it in a closure, and return the enhanced version — the `@decorator` syntax being nothing more than `func = decorator(func)`. You built banner and timing decorators, discovered how a plain wrapper destroys `__name__` and `__doc__` (fixed by `functools.wraps`), and unpacked parameterised decorators into their three-layer onion. Namespaces explain *where* names live; decorators show how that machinery lets you compose behaviour.

PYTHON

```

class Lion:
    __water_source="well in the circus"

    def __init__(self,name, gender):
        self.__name=name
        self.__gender=gender

    def drinks_water(self):
        print(self.__name,
              "drinks water from the",Lion.__water_source)

    @staticmethod
    def get_water_source():
        return Lion.__water_source

simba=Lion("Simba","Male")
simba.drinks_water()
print( "Water source of lions:",Lion.get_water_source())

```

OOP Part 2 — Encapsulation & Static

1. Modeling a Problem with Classes

1.1 Point and Line

Object-Oriented Programming shines when you name the real things in your problem and give each one its data plus its behavior. For the geometry scenario, the nouns are **Point** and **Line**.

- `Point` stores its coordinates (`x_cod`, `y_cod`) as instance attributes.
- `__str__` makes `print(p1)` show `<1,10>` instead of a memory address.
- `euclidean_distance(other)` applies the distance formula $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$ between two points.
- `distance_from_origin()` computes $\sqrt{x^2 + y^2}$ — the commented alternative `return self.euclidean_distance(Point(0,0))` reuses one method inside another.

`Line` stores the coefficients `A`, `B`, `C` of the line $Ax + By + C = 0$. `point_on_line(line, point)` plugs the point's coordinates into the line equation and checks whether the result is zero. `shortest_distance(line, point)` applies the point-to-line formula $|A*x + B*y + C| / \sqrt{A^2 + B^2}$.

Notice that these two methods name their parameters `line` and `point` instead of the conventional `self` — it still works because the instance is simply passed as the first argument, but later sections show the idiomatic `self` style. The payoff of this design: the main code collapses to `l1 = Line(1,1,-2)`, `p1 = Point(1,10)`, and a call such as `l1.shortest_distance(p1)` → `6.36`.

PYTHON

```

class Point:

    def __init__(self,x,y):
        self.x_cod = x
        self.y_cod = y

    def __str__(self):
        return '<{},{}>'.format(self.x_cod,self.y_cod)

    def euclidean_distance(self,other):
        return ((self.x_cod - other.x_cod)**2 + (self.y_cod - other.y_cod)**2)**0.5

    def distance_from_origin(self):
        return (self.x_cod**2 + self.y_cod**2)**0.5
        # return self.euclidean_distance(Point(0,0))

class Line:

    def __init__(self,A,B,C):
        self.A = A
        self.B = B
        self.C = C

    def __str__(self):
        return '{}x + {}y + {} = 0'.format(self.A,self.B,self.C)

    def point_on_line(line,point):
        # ...
        return "does not lie on the line"

    def shortest_distance(line,point):
        return abs(line.A*point.x_cod + line.B*point.y_cod + line.C)/((line.A**2 + line.B**2)**0.5)

```

PYTHON

```

l1 = Line(1,1,-2)
p1 = Point(1,10)
print(l1)
print(p1)

l1.shortest_distance(p1)

```

2. Accessing Object Attributes & Methods

2.1 Attributes are data, methods are actions

Once an object exists, you read its pieces and call its powers with the **dot operator**:

- `obj.attribute` reads a piece of data — the dot reaches into the object's namespace.
- `obj.method()` runs a function bound to the object — the parentheses are required.

- The constructor stored the values into `self` inside `__init__`; the dot retrieves them.
- Different objects of the same class return different attribute values — state is per-instance.
- Attributes are public by default in Python.
- Forgetting the `()` on a method gives you the function object itself, not its result.

This is the everyday syntax of OOP — you will write `p.name` and `p.greet()` constantly from here on. Spot the `Person` class in the first cell, then try both accesses in the cells after it.

PYTHON

```
p = Person('nitish', 'india')
p.name      # ATTRIBUTE – data property, no brackets
p.greet()   # METHOD – callable action, brackets needed
```

How objects access attributes

Key ideas

Classes/objects model state+behavior. Encapsulation hides internals; abstraction focuses on essentials. Inheritance/polymorphism extend reuse.

PYTHON

```
class Person:
    def __init__(self, name_input, country_input):
        self.name = name_input
        self.country = country_input

    def greet(self):
        if self.country == 'india':
            print('Namaste', self.name)
        else:
            print('Hello', self.name)
```

2.2 Deep dive — attribute access mechanics

- Attribute: `obj.attribute` — a data property.
- Method: `obj.method()` — callable, needs brackets.
- The constructor stores the incoming values into `self`, the object's own attribute dictionary.
- The dot syntax accesses that namespace.
- A method without `()` returns a bound function reference (its identity), not the result.
- Outside access to attributes is public by default.

The `Person` class below greets with `Namaste` when the country is `india`, and `Hello` otherwise — a tidy example of behavior decided by the object's own state. Notice the constructor parameter names `name_input` and `country_input`: they avoid shadowing `self.name` and make the storage step self-documenting. Get comfortable associating each `obj.attribute` access with the line in `__init__` that created it.

PYTHON

```
# how to access attributes
p = Person('nitish','india')
```

PYTHON

```
p.name
```

3. Non-existent Attributes

3.1 You cannot read what the object never got

An object only carries the attributes you actually created for it — usually inside `__init__`:

If a class never set `self.gender`, then `p.gender` fails with an `AttributeError`, and the message tells you exactly which name is missing and on which class.

GUARD

WHAT IT DOES

```
hasattr(p, 'gender')
```

checks before accessing

```
getattr(p, 'gender', None)
```

safe read with a default

The fix is almost always: define that attribute in `__init__` first. This error is one of the most common beginner mistakes — whenever it appears, ask yourself "did I set this on `self`?". In the code cell right below, `p.gender` raises the error on purpose; notice how Python names both the class and the missing attribute in its message, so debugging is mostly a matter of reading the traceback.

PYTHON

```
p.gender # AttributeError: 'Person' object has no attribute 'gender'
```

Non-existent attribute

- An attribute that was not created with `self.` in `__init__` cannot be accessed
- `AttributeError: 'Person' object has no attribute 'gender'`
- Fix: define that attribute in the class's `__init__` first

PYTHON

```
p.gender # AttributeError! class ne ye attribute define hi nahi kiya
```

3.2 Deep dive — `AttributeError` in practice

- Accessing an attribute that was never set on the object raises `AttributeError`.
- The message names the exact class and attribute: `'Person' object has no attribute 'gender'`.
- The object only holds what `__init__` (or a later assignment) created.
- Defensive reading: `hasattr(p, 'gender')` first, or `getattr(p, 'gender', None)`.

- Attributes are created at runtime — Python objects are dynamic.
- "Set-before-access" is the typical bug; read the class and the assignment line when debugging.

Since `gender` was never stored on `self`, the cell below raises the error on purpose — a small, instructive crash that makes the rule unforgettable. The error also teaches the language of tracebacks: read the bottom line first — `AttributeError`, then the object type, then the exact missing name. Nine times out of ten the fix is one line back in `__init__`, where the attribute should have been created.

PYTHON

```
# what if i try to access non-existent attributes
p.gender
```

PYTHON

```
p.gender # AttributeError: 'Person' object has no attribute 'gender'
```

4. Creating Attributes from Outside

4.1 Python objects are flexible

Unlike strict languages, Python lets you bolt new attributes onto an object from outside its class:

- The assignment simply adds `gender` to *that particular object's* namespace.
- Only that object changes — other instances are untouched.
- Nothing stops you; there is no compile-time check.

APPROACH

EFFECT

Inside `__init__`

consistent, guaranteed for every object

Outside (ad-hoc)

only for one object — handy for prototyping

Best practice: define all normal attributes in `__init__` for consistency, and save outside assignment for quick experiments and optional flags. You will add `p.gender = 'male'` in the cells below and see it come back instantly — which raises a design question worth answering now: *should* a class's every field be declared up front? For consistency, yes; for prototyping, the dynamic way is often faster.

PYTHON

```
p.gender = 'male'      # created AFTER construction
print(p.gender)        # 'male'
```

Creating an attribute from outside the class

- The Python objects are flexible — attributes can be added to an object dynamically
- This only affects that particular object
- For consistency, define all attributes in `__init__`

PYTHON

```
p.gender = 'male' # class ke bahar se object pe attribute add
print(p.gender)  # 'male'
```

4.2 Deep dive — the dynamic instance namespace

- Python objects are dynamic: `obj.new_attr = value` updates the instance namespace from anywhere.
- There is no real encapsulation by default — everything is public at runtime.
- The change affects ONLY that object, not other instances.
- Because it was never declared in `__init__`, it is an *optional* field.
- Use it for ad-hoc flags and quick prototyping.
- Risk: inconsistency — two objects of the same class can disagree on which attributes they have.
- Mental model: every instance has its own attribute dictionary you are free to edit.

Run the two cells below: `p.gender = 'male'` adds the attribute, and `p.gender` returns it. The class definition never mentioned `gender` — Python simply stored it on this one object. This is the flip side of Python's famous "everything is public": total freedom, and the risk that two instances drift apart. Decide deliberately — declared fields in `__init__` for the stable contract, dynamic attributes for quick experiments — and your code keeps both virtues.

PYTHON

```
p.gender = 'male'
```

PYTHON

```
p.gender
```

5. Reference Variables

5.1 The name is a pointer, not the object

When you create an object, Python sets aside memory for it; the variable is just a *label* attached to that memory:

- A reference variable does not contain the object — it holds its address.
- An object created without a reference (`Person()` with nothing on the left) becomes unreachable and is garbage-collected.
- One object can wear many labels: `q = p` creates NO new object, just a second name for the same one.
- Changing `q.name` changes what `p` sees, because both names reach the same memory.

This is why `id(p) == id(q)` is `True` after `q = p` — the label swap did not stamp out a copy.

PYTHON

```
p = Person()    # object created, labeled `p`
q = p          # q points at the SAME object
print(id(p), id(q)) # identical addresses
```

Reference Variables

- Reference variables hold the objects
- We can create objects without reference variable as well
- An object can have multiple reference variables
- Assigning a new reference variable to an existing object does not create a new object
- An object created without a reference gets lost in memory
- An object can have multiple reference variables
- `q = p` does NOT create a new object — same object, 2 labels
- Multiple refs → `print(id(p))`, `print(id(q))` give the same address

PYTHON

```
class Person:
    def __init__(self):
        self.name = 'nitish'
        self.gender = 'male'

p = Person()    # object is referenced by label `p`
q = p          # q bhi same object ko point karega!
```

5.2 Deep dive — object lifecycle & multiple references

- `Person()` without assignment → the object is born, but no label keeps it alive.
- Reference count drops to 0 → **garbage collected** immediately after `'Created'` prints.
- A reference variable is a tag pointing at the object.
- Multiple references: `q = p` — same object, two names; `id(p) == id(q)` confirms it.
- Changing an attribute through `q` is visible through `p` — same underlying object.
- Rebinding a label (`p = something_else`) redirects only that name; the old object lives while another label still points to it.
- Python manages memory for you; later you may meet `del`, `weakref`, and the `gc` module for edge cases.

The cells below create a `Person`, alias it as `q`, print matching ids, and then change `q.name` — demonstrating that `p` sees the change too to `ankit`.

PYTHON

```
# object without a reference
class Person:

    def __init__(self):
        self.name = 'nitish'
        self.gender = 'male'

p = Person()
q = p
```

PYTHON

```
# Multiple ref
print(id(p))
print(id(q))
```

Deep Dive : Multiple references — same object

- `q = p` — assignment of a REFERENCE, not a copy
- Both names → the same memory
- Change via `q` → visible on `p` (same underlying object)
- id verification: `id(p) == id(q)` True
- This is the fundamental difference between primitive-immutable and object-mutable
- If you don't want sharing — use `copy()`, `deepcopy()`
- Aliasing concept — happens more with collections of objects
- Interview: "does p change when q changes?"

PYTHON

```
print(p.name)
print(q.name)
q.name = 'ankit'
print(q.name)
print(p.name)
```

PYTHON

```
p = Person('nitish')      # 1 object
q = p                    # q same object ko point
id(p) == id(q)           # True — SAME object
q.name = 'ankit'         # use object ka name change
print(p.name)            # 'ankit' — p bhi dekh Lega!
```

6. Pass by Reference

6.1 What goes into the function, comes out changed

Functions receive objects **by reference** — the same memory, not a copy:

- Object parameters are references; attribute changes inside the function leak outside.
- Immutables (int, str, tuple) get *rebound*, so reassigning a parameter inside the function never affects the caller.
- Mutables (list, dict, your objects) get *mutated*, so in-place changes are visible outside.
- If the caller must stay unchanged, pass or work on a copy.

The root cause is the aliasing you already saw with `q = p` — labels `p` and `person` point to one shared object, now crossing a function boundary. The cells below demonstrate both flavours: one function reads the object and builds a brand-new one to return, while another mutates the passed-in object, proving the distinction between "outside" data and "shared" data.

PYTHON

```
def greet(person):
    person.name = 'ankit'  # mutates the caller's object
    print(person.name)

p = Person('nitish', 'male')
greet(p)                  # prints 'ankit'
print(p.name)             # also 'ankit' — the original changed!
```

Pass by reference — the change inside is visible outside

- In Python objects are passed **by reference**, not by copy
- Attribute change inside the function → visible on the caller side too
- Strings/immutable get copied, objects do not

PYTHON

```
def greet(person):          # person object ka reference hai
    person.name = 'ankit'  # original object change!
    print(person.name)

p = Person('nitish', 'male')
greet(p)
print(p.name)              # 'ankit' — same object andar gya
```

7. Object Mutability

7.1 Mutable objects change in place

Because objects are mutable, any method or function holding a reference can modify them:

- `person.name = ...` modifies the object no matter which reference performed the write.
- The function's `return person` hands back the SAME object, so `q` and `p` point to one thing.
- This mirrors list behavior you already know: `append` inside a function changes the list outside.

The opposite is immutability: an integer like `5` can never change in place — `x = x + 1` rebinds `x` to a brand-new value, leaving the old one alone. Keep that contrast in your mental model: immutables are replaced, mutables are edited — and your objects belong firmly to the editable camp.

PYTHON

```
def greet(person):
    person.name = 'ankit'  # edits the shared object
    return person

p = Person('nitish', 'male')
q = greet(p)
print(p.name, q.name)    # both 'ankit' - same object
```

7.2 Deep dive — mutable state everywhere

- Objects are **mutable** — methods and functions can modify them in place.
- `self.name = ...` works on the same object no matter which method or function runs.
- `return person` returns a reference, not a copy — `p` and `p1` share memory.
- The two ids print equal, confirming identity.
- This is the same feature you used when mutating lists.

Run the cell and watch `id(p) == id(p1)`. If you ever want independent copies, reach for `copy()` or `copy.deepcopy()` — topics for the collections chapters. Mutation and side effects are not bugs by themselves — they are how objects cooperate — but they demand awareness: if a function unexpectedly rewrites your data, ask who else holds a reference. Aliasing is the root cause of a whole family of "why did it change?" mysteries.

PYTHON

```
class Person:

    def __init__(self, name, gender):
        self.name = name
        self.gender = gender

# outside the class -> function
def greet(person):
    person.name = 'ankit'
    return person

p = Person('nitish', 'male')
print(id(p))
p1 = greet(p)
print(id(p1))
```

PYTHON

```
def greet(person):
    person.name = 'ankit'
    return person

p = Person('nitish', 'male')
p1 = greet(p)
print(id(p))      # some address
print(id(p1))     # SAME address – the function returned the same object
```

8. Encapsulation

8.1 Private data behind public gates

Encapsulation hides an object's internal state behind methods so outside code cannot pounce on it directly:

- `__name` (double underscore) marks an attribute **private** via name mangling.
- Direct access from outside fails; code must use the methods.
- Getters read, setters write, and setters can **validate** — e.g. accept only integers.
- Benefits: change internal logic later without breaking callers, and keep state consistent (invariants).

Privacy is a strong convention, not a hard wall — the mangled name `_Atm__balance` still technically works, but touching it is discouraged. Enforce the contract with a public method and private storage; the reader of any other class then knows instantly which names are "for internal use" without reading documentation.

PYTHON

```
class Atm:
    def __init__(self):
        self.pin = ''      # public attribute
        self.__balance = 0 # private – __ prefix

    def get_balance(self):
        return self.__balance # gatekeeper (read)

    def set_balance(self, value):
        if type(value) == int:
            self.__balance = value # gatekeeper (write, with validation)
```

8.2 Deep dive — name mangling & private members

- `__var` (double underscore) marks an attribute **private**; Python *name-mangles* it.
- What is really stored is `_Atm__pin` — technically reachable, but by convention private.
- Getter to read (`get_balance`), setter to write (`set_balance`).
- The setter can validate: here, a pin must be four digits.
- Encapsulation protects invariants — an account balance should never be changed at will.
- Public interface = methods; private = data.

- Not enforced by the language (unlike Java `private`) — it is a strong convention.

The `Atm` class in the cells below puts it into practice with a guarded balance and validate-before-write setters. Start with your own raw `__atm_pin` readable, too — then comment it out and watch the `AttributeError` appear: the mangling is doing its job the moment the name stops resolving.

PYTHON

```
# instance var -> python tutor
class Person:

    def __init__(self, name_input, country_input):
        self.name = name_input
        self.country = country_input

p1 = Person('nitish', 'india')
p2 = Person('steve', 'australia')
```

PYTHON

```
p2.name
```

8.3 The guarded Atm at work

The full `Atm` in the upcoming cells wraps its balance in `__balance` and exposes `get_balance()` / `set_balance()`. `set_balance` refuses non-integer values — validation lives where the write happens. The menus, pin creation and withdrawal logic all work through the same private state, so no outside code can corrupt the balance.

The test cells that follow do the essentials: `obj = Atm()` prints the object id from inside the constructor; `obj.get_balance()` reads through the getter; `obj.set_balance(1000)` writes through the validating setter; and the interactive `obj.withdraw()` asks for the pin and amount, then deducts from the private `__balance` when the amount is affordable. What matters is the pattern — private state plus public, validated behavior.

PYTHON

```

class Atm:

    # constructor(special function)->superpower ->
    def __init__(self):
        print(id(self))
        self.pin = ''
        self.__balance = 0
        #self.menu()

    def get_balance(self):
        return self.__balance

    def set_balance(self, new_value):
        if type(new_value) == int:
            self.__balance = new_value
        else:
            print('beta bahot maarenge')

    def __menu(self):
        user_input = input("""
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
""")

        # ...
        else:
            print('abe garib')
        else:
            print('sale chor')

```

PYTHON

```
obj = Atm()
```

9. Collection of Objects

9.1 Objects love living in lists and dicts

Real systems create hundreds of objects; the natural next step is storing them in the containers you already know:

- A **list of objects** suits sequential processing — loops, filters, simulations.
- A **dict of objects** supports fast lookup by id or name.
- Inside the loop every element is a normal object: `p.name`, `p.greet()`.
- List comprehensions work too: `[p.name for p in L]`.

- Sorting by attribute: `sorted(L, key=lambda p: p.name)`.

Storing objects in containers is what turns OOP from "one class" into a working little system.

PYTHON

```
p1 = Person('nitish', 'male')
p2 = Person('ankit', 'male')
p3 = Person('ankita', 'female')

L = [p1, p2, p3]          # List of objects
for p in L:
    print(p.name, p.gender) # iterate and use each object

d = {'p1': p1, 'p2': p2}  # dict of objects
for k in d:
    print(d[k].gender)    # access by key
```

Collection of objects

- Keep objects in lists/dicts → use their methods/attributes
- Common: store database/batch data as objects and iterate over them
- During iteration `p.name`, `p.gender` are accessible on every element

PYTHON

```
L = [p1, p2, p3]          # List of Person objects
for p in L:
    print(p.name)

d = {'1': p1, '2': p2}    # dict of objects bhi
```

9.2 Deep dive — structuring many objects

- Real systems hold hundreds of objects → organize them in a structure.
- List of objects: sequential iterate/process — records, simulations.
- Dict of objects: lookup by key (id, name, email).
- The loop accesses each element's attributes and methods directly.
- Comprehension trick: `{p.name: p for p in L}` builds a dict keyed by an attribute.
- `sorted(L, key=lambda p: p.name)` orders objects by an attribute.
- Grouping data and behavior into containers is collection handling — a core daily skill.

The cells below build a `Person` list and loop to print names and genders, then a `Person` dict and loop over its keys — both styles in action.

PYTHON

```
# List of objects
class Person:

    def __init__(self,name,gender):
        self.name = name
        self.gender = gender

p1 = Person('nitish','male')
p2 = Person('ankit','male')
p3 = Person('ankita','female')

L = [p1,p2,p3]

for i in L:
    print(i.name,i.gender)
```

PYTHON

```
# dict of objects
# list of objects
class Person:

    def __init__(self,name,gender):
        self.name = name
        self.gender = gender

p1 = Person('nitish','male')
p2 = Person('ankit','male')
p3 = Person('ankita','female')

d = {'p1':p1,'p2':p2,'p3':p3}

for i in d:
    print(d[i].gender)
```

10. Static Variables vs Instance Variables

10.1 Class-level vs object-level state

Every object carries its own copy of **instance variables** — but sometimes a value should be shared by the whole class:

KIND	WHERE	COPIES	REACH
Instance var	set with <code>self.</code> in <code>__init__</code>	one per object	<code>obj.var</code>
Static var	set at class level	one shared	<code>ClassName.var</code>

- Static variables exist once per class; every instance sees the same value.

- Access them with the class name: `Atm.__counter`, even with no object.
- They are perfect for counters (how many objects have been made), config values, and shared constants.
- The pattern above gives every new object the next id: read the counter, store it as `cid`, then bump it.

PYTHON

```
class Atm:
    __counter = 1      # static/class variable – ONE copy for all instances

    def __init__(self):
        self.pin = ''  # instance variable – every object's own copy
        self.cid = Atm.__counter
        Atm.__counter += 1
```

Static Variables(Vs Instance variables)

What/Why

Variables reference objects in memory. Data types determine operations allowed and behavior.

Semantics

Python is dynamically typed. Assignment binds names to objects; mutability matters.

Common mistakes

Confusing assignment vs comparison, mutability side effects.

Static variables (vs instance)

- **Instance variable** — every object has its own copy (`self.pin`)
- **Static variable** — class level (`Atm.__counter`), shared by all objects
- Static access: `ClassName.variable` (works even without an object)
- Use-case: counter, config, shared global value

PYTHON

```
class Atm:
    __counter = 1      # static: class level, sab objects share

    def __init__(self):
        self.pin = ''  # instance: har object ki copy
        self.cid = Atm.__counter
        Atm.__counter += 1
```

10.2 Deep dive — the counting Atm

- **Static/class variable:** class-level, one copy shared by everyone.
- **Instance variable:** each object's own separate copy.

- Use statics for counters, config, shared state.
- Access via `ClassName.var` or `self.var` (both reach the same slot) — statics work even with NO instance.
- Auto-increment pattern: read the shared counter, assign it to the new object as `cid`, then bump the counter.

The far-reaching `Atm` class in the cells uses exactly this: it stores a serial id (`cid`) on every object and also exposes a `@staticmethod` `get_counter()`. The saved outputs show `Atm.get_counter()` → 2 and `c3.cid` → 3 — the precise numbers depend on how many objects were created before that snapshot, but the mechanism stays the same: one shared, ever-increasing class counter.

PYTHON

```
class Atm:

    __counter = 1

    # constructor(special function)->superpower ->
    def __init__(self):
        print(id(self))
        self.pin = ''
        self.__balance = 0
        self.cid = Atm.__counter
        Atm.__counter = Atm.__counter + 1
        #self.menu()

    # utility functions
    @staticmethod
    def get_counter():
        return Atm.__counter

    def get_balance(self):
        return self.__balance

    def set_balance(self, new_value):
        if type(new_value) == int:
            self.__balance = new_value
        else:
            print('beta bahot maarenge')

    # ...
    else:
        print('abe garib')
    else:
        print('sale chor')
```

PYTHON

```
c1 = Atm()
```

11. Static Methods

11.1 Behavior that needs no object

A **static method** belongs to the class but expects no `self`:

- The `@staticmethod` decorator marks a method that does not receive `self`.
- Call it through the class: `Lion.get_water_source()` — no instance needed.
- Perfect for helper or utility logic that does not depend on any object's state.
- It can still read static variables via `ClassName.var`.

METHOD KIND	FIRST ARG	NEEDS AN INSTANCE?
Instance method	<code>self</code>	yes
<code>@staticmethod</code>	none	no
<code>@classmethod</code>	<code>cls</code>	no (receives the class)

Static methods are "pure functions grouped with the class" — handy utilities living inside the class's namespace. The cells below show the triple life of every member: instance methods (need `self`), class methods (need `cls` for shared state), and static methods (need nothing) — keep their varargs as the tell that decides which world a method lives in.

PYTHON

```
class Lion:
    __water_source = "well in the circus"  # static variable

    @staticmethod
    def get_water_source():                # static method – no self
        return Lion.__water_source
```

Points to remember about static

- Static attributes are created at class level.
- Static attributes are accessed using `ClassName`.
- Static attributes are object independent. We can access them without creating instance (object) of the class in which they are defined.
- The value stored in static attribute is shared between all instances(objects) of the class in which the static attribute is defined.

Static methods & static reminders

- Static attributes are created at class level — no `obj` involved
- Access statics with `ClassName.attr`
- Use the private static `__water_source` only from inside the class

- Meaning of the "points to remember" block: object-independent, shared, class scope

PYTHON

```
class Lion:
    __water_source = 'well in the circus' # static private

    def __init__(self, name, gender):
        self.__name = name

    def drinks_water(self):
        print(self.__name, 'drinks from', Lion.__water_source)
```

Deep Dive : Static methods

- `@staticmethod` — a method without `self`
- Called via the class: `Lion.drink()` — no instance needed
- When to use: helper logic that does not depend on state
- A utility living inside the class namespace
- No access to `self` / instance data
- Can access static vars via the `ClassName`
- vs instance method — `self` is required
- vs classmethod `@classmethod` (`cls` as the first parameter)
- Statics: pure functions grouped with the class
- Point to remember: static attrs are created at class load time

PYTHON

```
class Lion:
    __water_source = "well in the circus" # static var
    @staticmethod
    def drink(): # static method
        print("Drinking from", Lion.__water_source)
```

OOP Part 3 — Inheritance & Polymorphism

1. Class Relationships (Has-A / Is-A)

1.1 Aggregation — the Has-A relationship

Objects rarely live alone; more often one object *contains* another:

- **Aggregation (Has-A)** — a class holds an object of another class as one of its attributes.
- A `Customer` has an `Address`; an `Employee` has a `Department`; a `Car` has an `Engine`.
- The contained object usually exists on its own (an address makes sense without a customer) — this loose coupling makes aggregation flexible.
- You reach inside with the dot chain: `cust.address.city`.

- Class diagrams draw it with a hollow diamond on the container side.

The customer example coming up also shows methods delegating to the contained object: `edit_profile` updates the address by calling `self.address.edit_address(...)` — one object asking another to take care of itself.

```
PYTHON

class Customer:
    def __init__(self, name, gender, address):
        self.name = name
        self.gender = gender
        self.address = address  # Customer HAS-A Address
```

1.2 Has-A vs Is-A — choose the right relationship

DECISION	RELATIONSHIP	HOW	MEANING
Does one thing <i>contain</i> another?	Aggregation (Has-A)	nested attribute	<code>Customer</code> has <code>Address</code>
Is one thing a <i>kind of</i> another?	Inheritance (Is-A)	<code>class Child(Parent)</code>	<code>Student</code> is a <code>User</code>

- Has-A → composition/containment; one object stores another.
- Is-A → specialization; the child inherits and extends the parent.
- Aggregation keeps classes independent and reusable; inheritance couples them through a hierarchy.
- Rule of thumb: prefer aggregation ("uses") unless there is a genuine "is a kind of" relationship.

This session explores both relationships, starting with the aggregation code cells below. In short: choose Has-A when one object *uses* another, and Is-A when one object is a more specific version of another. Getting that decision right keeps your hierarchy honest and your code easy to extend.

```
PYTHON

# Aggregation : Customer HAS-A Address
class Customer:
    def __init__(self, name, address):
        self.name = name
        self.address = address

# Inheritance : Student IS-A User
class Student(User):
    pass
```

Deep Dive : Aggregation — Has-A relationship

- **Aggregation (Has-A)** — an object of another class inside a class
- Customer HAS-A Address — relation by containment
- The part can be independent (an address can exist without the customer)
- One-directional reference

- vs inheritance (Is-A) — compared further below
- Real: Order→Items, Car→Engine, User→Address
- Access: `cust.address.city`
- Object graphs — real-world modeling
- Composition = strong aggregation (the part cannot live alone)

PYTHON

```
# example
class Customer:

    def __init__(self, name, gender, address):
        self.name = name
        self.gender = gender
        self.address = address

    def print_address(self):
        print(self.address.__city, self.address.pin, self.address.state)

    def edit_profile(self, new_name, new_city, new_pin, new_state):
        self.name = new_name
        self.address.edit_address(new_city, new_pin, new_state)

class Address:

    def __init__(self, city, pin, state):
        self.__city = city
        self.pin = pin
        self.state = state

    def get_city(self):
        return self.__city

    def edit_address(self, new_city, new_pin, new_state):
        self.__city = new_city
        self.pin = new_pin
        # ...

cust.edit_profile('ankit', 'mumbai', 111111, 'maharashtra')
cust.print_address()
# method example
# what about private attribute
```

PYTHON

```
class Customer:
    def __init__(self, name, gender, address):
        self.name = name
        self.gender = gender
        self.address = address    # Address OBJECT inside Customer
```

2. Inheritance — the Is-A Relationship

2.1 One class wearing another's clothes

Inheritance creates a parent-child relationship between classes:

- Syntax: `class Child(Parent):`.
- The child **receives** the parent's non-private attributes and methods — it can use them as its own.
- **Is-A**: a Student Is-A User.
- Benefits of inheritance:
 - **Code reuse** — you never rewrite the parent's logic.
 - **Extension** — the child adds its own methods and attributes on top.
 - **Overridability** — the child replaces parent methods, the seed of polymorphism.
 - **Hierarchy** — modeling the world as families (Vehicle → Car → ElectricCar).

The most common confusion: inherited members become *accessible*, not *mandatory* — the child decides what to use, override, or extend. Think of the child as the parent plus a few extra powers: it can use the inherited rooms (attributes and methods), owns new rooms of its own, and may remodel a kitchen (override a method) without tearing down the architecture.

PYTHON

```
class User:                # parent / base class
    def __init__(self):
        self.name = 'nitish'

class Student(User):       # child / derived class
    pass
```

Inheritance

- What is inheritance
- Example
- What gets inherited?

Inheritance (Is-A relationship)

- **Inheritance** = "Is-A" — Student is a User
- Benefits: code reuse, hierarchy, overridability
- The child takes the parent's things (constructor, public attrs, public methods) into its own possession — but it does not force them; it can access them

PYTHON

```
class User:                                # parent / base class
    def __init__(self):
        self.name = 'nitish'

class Student(User):                       # child class User se inherit
    pass
```

2.2 Deep dive — method overriding

- **Overriding** = the child redefines a parent method with the same name.
- Call resolution searches the child class first, then walks up; the child's version wins.
- To keep the parent logic too, call `super().m1()` from inside the child's version.
- Matching argument lists makes overrides clean and predictable.
- Real example: a `Phone` buys as a phone; a `SmartPhone` overrides `buy()` to announce a smartphone (as you will see in the code).
- Overriding is the foundation of polymorphism.
- Python does not do *overloading* by default — the last definition of a name wins. The chain of lookups explains the mechanism: Python checks the object's class first, then walks the parents in MRO order — an override behaves like a closer definition in a scoping chain, just at the class level.

PYTHON

```
class A:
    def __init__(self):
        self.var1 = 100
    def m1(self):
        print("Method in A")

class B(A):
    def m1(self):                # OVERRIDE — same name, own version
        print("Method in B")

b = B()
b.m1()                          # "Method in B" — the child's version wins
```

2.3 Deep dive — what the child inherits

Three rules are visible here:

- **Constructor:** because `Student` defines its own `__init__`, calling `Student()` runs only the child's — `rollno` is set, but `name` / `gender` are NOT, unless the child calls `super().__init__()`.
- **Methods:** `login()` was inherited, and the child wrote `enroll()` — `s.login()` and `s.enroll()` both work.
- **Snapshot honesty:** the cell's saved output shows `nitish` because it was captured when `Student` had NO `__init__` of its own. Run the current source fresh and `print(s.name)` would raise `AttributeError` — a live reminder that *the constructor decides which attributes exist*.

That is exactly why the accompanying `Phone` / `SmartPhone` examples stress calling `super().__init__(...)` when a child wants both its own fields and the parent's.

PYTHON

```
# Example

# parent
class User:

    def __init__(self):
        self.name = 'nitish'
        self.gender = 'male'

    def login(self):
        print('login')

# child
class Student(User):

    def __init__(self):
        self.rollno = 100

    def enroll(self):
        print('enroll into the course')

u = User()
s = Student()

print(s.name)
s.login()
s.enroll()
```

PYTHON

```
class User:
    def __init__(self):
        self.name = 'nitish'
        self.gender = 'male'
    def login(self):
        print('login')

class Student(User):
    def __init__(self):          # own constructor → parent's is NOT auto-run
        self.rollno = 100
    def enroll(self):
        print('enroll into the course')
```

What gets inherited?

- Constructor
- Non Private Attributes
- Non Private Methods

THING	INHERITED?
Constructor (<code>__init__</code>)	✓ (if the child has none)
Non-private attributes	✓
Non-private methods	✓
Private members (<code>__name</code>)	✗

- If the child does not define `__init__`, the parent's comes in by default
- If the child defines its own `__init__`, it overrides the parent's
- A private `__member` can never be accessed directly in the child

PYTHON

```
class Phone:
    def __init__(self, price, brand, camera):
        print('Inside phone constructor')
        self.price = price
        self.brand = brand
        self.camera = camera
```

2.4 Deep dive — constructor inheritance rules

THING	INHERITED?
Constructor (<code>__init__</code>)	only if the child defines none; otherwise call <code>super().__init__()</code>
Non-private attributes/methods	✓
Private members (<code>__x</code>)	✗
Class/static variables	✓

- Child with NO `__init__` → the parent's constructor is used automatically (via MRO).
- Child with its own `__init__` → the parent's is NOT run unless you call `super().__init__()`.
- Public attributes (`brand`, `camera`) come along; the private `__price` is mangled to `Phone` only, so the child cannot touch it — but a public getter like `show()` inside the parent CAN read it.
- Python resolves lookups through **MRO**, the Method Resolution Order.

The code cells after this demonstrate exactly these cases: a child that inherits the constructor, a child that replaces it, and a child trying to reach the parent's private `__price`.

PYTHON

```
# constructor example

class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class SmartPhone(Phone):
    pass

s=SmartPhone(20000, "Apple", 13)
s.buy()
```

PYTHON

```
# constructor example 2

class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

class SmartPhone(Phone):
    def __init__(self, os, ram):
        self.os = os
        self.ram = ram
        print ("Inside SmartPhone constructor")

s=SmartPhone("Android", 2)
s.brand
```

3. Method Overriding

3.1 The child rewrites the parent

When a child defines a method with the same name as the parent's, it **overrides** it:

- Overriding = same method name, the child supplies its own version.
- Python resolves the call at runtime by walking up the class chain (child first) — this is **dynamic dispatch**.
- The child's version fully replaces the parent's for that object.
- If the child still wants the parent's behavior too, it calls `super().buy()`.

You will see two further shapes in the code cells: a child that *adds a new method* (`display2`) without overriding — so the parent's `display1` still runs — and a `Phone` / `SmartPhone` pair where the child overrides `buy()` and prints its own message.

PYTHON

```
class Phone:
    def buy(self):
        print("Buying a phone")

class SmartPhone(Phone):
    def buy(self):                # override
        print("Buying a smartphone")

s = SmartPhone(20000, "Apple", 13)
s.buy()                         # "Buying a smartphone"
```

Method overriding

- Same method name in the child as in the parent → **overriding**
- At runtime the child's version runs (part of polymorphism)
- If you also want the parent's, call `super().method()`

PYTHON

```
class A:
    def __init__(self):
        self.var1=100

    def display1(self,var1):
        print("class A :", self.var1)
class B(A):
    def display2(self,var1):
        print("class B :", self.var1)

obj=B()
obj.display1(200)
```

PYTHON

```
# Method Overriding
class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class SmartPhone(Phone):
    def buy(self):
        print ("Buying a smartphone")

s=SmartPhone(20000, "Apple", 13)

s.buy()
```

4. The Super Keyword

4.1 Reaching back to the parent

`super()` is Python's reference *from the child up to the parent*:

- `super().__init__(...)` invokes the parent's constructor explicitly.
- `super().method()` calls a parent method you overrode, e.g. `super().buy()`.
- Use it when the child wants BOTH the parent's setup AND its own extras.
- `super` can reach the parent's public and protected members, but not its private mangled names.
- It is meaningful only inside a child's methods — there is no sensible `super()` outside the class body.

The pattern to internalize: **override, then extend** — in the child's version, do your own work, then call `super().whatever()` to keep the parent's behavior flowing. A useful test: if you catch yourself copy-pasting the parent's `__init__` body into the child, stop — that is exactly the duplication `super()` exists to remove. Write the child's extras, and one line brings in everything the parent already does.

PYTHON

```
class SmartPhone(Phone):
    def __init__(self, price, brand, camera, os):
        super().__init__(price, brand, camera) # parent constructor
        self.os = os
```

Super Keyword

PYTHON

```
class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class SmartPhone(Phone):
    def buy(self):
        print ("Buying a smartphone")
        # syntax to call parent ka buy method
        super().buy()

s=SmartPhone(20000, "Apple", 13)

s.buy()
```

The Super keyword

- `super()` = a reference to the parent class
- `super().__init__(...)` — calls the parent's constructor explicitly
- Use-case: the child wants both its own extra attributes and the parent's setup
- `super` can also access the parent's data (through methods)

PYTHON

```
class Phone:
    def __init__(self, price, brand, camera):
        print('Inside phone constructor')
        self.price = price
        self.brand = brand
        self.camera = camera

class SmartPhone(Phone):
    def __init__(self, price, brand, camera, os):
        super().__init__(price, brand, camera) # parent constructor ko call
        self.os = os
```

4.2 Deep dive — super in the constructor chain

Call order for `SmartPhone(20000, "Samsung", 12, "Android", 2)`:

1. `SmartPhone.__init__` starts and prints 'Inside smartphone constructor'.
2. `super().__init__(...)` runs `Phone.__init__`, printing 'Inside phone constructor' and setting `__price`, `brand`, `camera`.

3. Control returns; `os` and `ram` are set on the child.

4. `s.os` → 'Android', `s.brand` → 'Samsung' — the public attributes are reachable.

- `super()` is MRO-aware: it resolves to the next class in the hierarchy.
- Use it in constructor chaining and when extending an overridden method.
- A `super()` call outside the class body has no meaning.
- Private parent data stays off-limits even through `super()` (name mangling binds it to the parent).

A small note: a cell that attempts `print(super().brand)` would raise `AttributeError` on a fresh run — `super()` resolves class-level lookups, while instance attributes like `brand` belong to `self` — so read instance data through `self`, and use `super()` for methods and constructor wiring.

PYTHON

```
# using super outside the class
class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class SmartPhone(Phone):
    def buy(self):
        print ("Buying a smartphone")
        # syntax to call parent ka buy method
        super().buy()

s=SmartPhone(20000, "Apple", 13)

s.buy()
```

PYTHON

```

# can super access parent ka data?
# using super outside the class
class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class SmartPhone(Phone):
    def buy(self):
        print ("Buying a smartphone")
        # syntax to call parent ka buy method
        print(super().brand)

s=SmartPhone(20000, "Apple", 13)

s.buy()

```

Inheritance in summary

- A class can inherit from another class.
- Inheritance improves code reuse
- Constructor, attributes, methods get inherited to the child class
- The parent has no access to the child class
- Private properties of parent are not accessible directly in child class
- Child class can override the attributes or methods. This is called method overriding
- `super()` is an inbuilt function which is used to invoke the parent class methods and constructor

PYTHON

```

class Parent:

    def __init__(self,num):
        self.__num=num

    def get_num(self):
        return self.__num

class Child(Parent):
    def __init__(self,num,val):
        super().__init__(num)
        self.__val=val

    def get_val(self):
        return self.__val

son=Child(100,200)
print(son.get_num())
print(son.get_val())

```

PYTHON

```

class Parent:
    def __init__(self):
        self.num=100

class Child(Parent):

    def __init__(self):
        super().__init__()
        self.var=200

    def show(self):
        print(self.num)
        print(self.var)

son=Child()
son.show()

```

5. Types of Inheritance

5.1 The family shapes of classes

Inheritance comes in a few standard shapes:

TYPE	STRUCTURE	EXAMPLE
Single	1 parent → 1 child	<code>class SmartPhone(Phone)</code>
Multilevel	grandparent → parent → child	<code>Product → Phone → SmartPhone</code>
Hierarchical	1 parent → many children	<code>Phone → SmartPhone, FeaturePhone</code>
Multiple	many parents → 1 child	<code>class SmartPhone(Phone, Product)</code>

- **Single** is the everyday case: one base, one child.
- **Multilevel** chains generations; the grandchild accumulates everything above it.
- **Hierarchical** shares one base between several siblings.
- **Multiple** imports behavior from several bases — powerful, but it introduces the **diamond problem**: when two parents share a common ancestor, whose method wins? Python's **MRO** (Method Resolution Order) answers deterministically, and `ClassName.__mro__` reveals the order.

PYTHON

```

class SmartPhone(Phone): pass           # single
class Phone(Product): pass              # multilevel (with SmartPhone)
class FeaturePhone(Phone): pass         # hierarchical
class SmartPhone(Phone, Product): pass  # multiple

```

Types of Inheritance

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance(Diamond Problem)
- Hybrid Inheritance

TYPE	MEANING
Single	1 parent → 1 child
Multilevel	parent → child → grandchild (chain)
Hierarchical	1 parent → many children
Multiple	many parents → 1 child

- Multiple → the diamond problem occurs (MRO decides which one runs)
- `ClassName.__mro__` — in which order the parent classes are looked up

PYTHON

```

class Phone: ...           # parent
class SmartPhone(Phone): ... # single

class Product: ...         # multilevel
class Electronic(Product): ...
class Mobile(Electronic): ...

```

5.2 Deep dive — MRO and the diamond problem

- Multiple inheritance `class C(A, B)` — the plain lookup order is left-to-right.
- **MRO** (C3 linearization) resolves conflicts consistently; the order in the class list matters.
- Diamond problem: with `A → B, C → D`, if `B` and `C` both override `A.m1`, which version runs on `D`? The MRO keeps the order in which the classes were declared.
- `D.mro()` prints the full resolution order — the debugging tool for surprises.
- In the running cells, `class SmartPhone(Phone, Product)` with duplicate `buy` prints `Phone`'s version because `Phone` is first.
- Keep inheritance trees shallow, and prefer composition over multiple inheritance in serious designs.

PYTHON

```

# single inheritance
class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class SmartPhone(Phone):
    pass

SmartPhone(1000, "Apple", "13px").buy()

```

PYTHON

```
# multilevel
class Product:
    def review(self):
        print ("Product customer review")

class Phone(Product):
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class SmartPhone(Phone):
    pass

s=SmartPhone(20000, "Apple", 12)

s.buy()
s.review()
```

Deep Dive : Diamond problem

- In multiple inheritance the same ancestor's method has 2 paths
- Conflict: which D.m1()?
- Python MRO (C3 linearization) solves it with a deterministic order
- `D.mro()` → check the order list
- `class D(B, C)` — B listed first → the B chain wins
- Ambiguity risk — the same base twice in the graph
- Fix: explicit MRO order, `super()` propagation
- Prefer composition over multiple inheritance (design advice)
- Interview classic — Python handles it quietly; other languages don't

PYTHON

```
# the diamond problem
# https://stackoverflow.com/questions/56361048/what-is-the-diamond-problem-in-python-and-why-its-not-appea

class Phone:
    def __init__(self, price, brand, camera):
        print ("Inside phone constructor")
        self.__price = price
        self.brand = brand
        self.camera = camera

    def buy(self):
        print ("Buying a phone")

class Product:
    def buy(self):
        print ("Product buy method")

# Method resolution order
class SmartPhone(Phone, Product):
    pass

s=SmartPhone(20000, "Apple", 12)

s.buy()
```

PYTHON

```
class A:

    def m1(self):
        return 20

class B(A):

    def m1(self):
        return 30

    def m2(self):
        return 40

class C(B):
    def m2(self):
        return 20

obj1=A()
obj2=B()
obj3=C()
print(obj1.m1() + obj3.m1()+ obj3.m2())
```

Diamond problem

- In multiple inheritance, when the same method exists in several parents → ambiguity

- Python's MRO (Method Resolution Order) decides which one runs
- `super()` looks up the parent's method according to the MRO
- Avoid recursion — `self.m1()` inside itself calls the same method again

PYTHON

```
class A:
    def m1(self):
        return 20

class B(A):
    def m1(self):
        val=super().m1()+30
        return val

class C(B):
    def m1(self):
        val=self.m1()+20
        return val
obj=C()
print(obj.m1())
```

PYTHON

```
class A:
    def m1(self):
        return 20

class B(A):
    def m1(self):
        val = super().m1() + 30
        return val

class C(B):
    def m1(self):
        val = self.m1() + 20    # warning: recursion risk!
        return val
```

6. Polymorphism

6.1 Many forms, one name

Polymorphism means "many forms" — the same method or operator behaves differently depending on the object it meets:

- **Overriding** is decided at runtime through dynamic dispatch.
- **Overloading**: Python has no real overloading — the last definition wins — so emulate it with default arguments (`area(2)` vs `area(3,4)`).

- **Operator overloading:** you met `__add__` in Session 7; `+` behaves per type.

The `Shape.area` cell prints `12.56` for a "circle" and `12` for a "rectangle" from ONE method — polymorphism at its simplest.

PYTHON

```
# 1. Method Overriding – child redefines a parent method
class SmartPhone(Phone):
    def buy(self): ...

# 2. Method Overloading – same name, different arguments
class Shape:
    def area(self, a, b=0): # one method, two call shapes
        return 3.14*a*a if b == 0 else a*b

# 3. Operator Overloading – the same + means different things
'hello' + 'world' # concatenation
4 + 5 # addition
[1, 2] + [3, 4] # merging
```

Polymorphism

- Method Overriding
- Method Overloading
- Operator Overloading

Polymorphism = **many forms** — the same method name does different work in different classes. Three types:

- `+` does its own work in every type — operator overloading
- In Python, method overloading is achieved with default values
- Overriding is decided at runtime — Python dynamic dispatch

PYTHON

```
class Shape: # 1. Method Overloading (default args se)
    def area(self, a, b=0):
        if b == 0:
            return 3.14 * a * a
        return a * b

# 2. Method Overriding (child parent ka method re-define)

# 3. Operator OverLoading
'hello' + 'world' # str: concat
4 + 5 # int: add
[1, 2] + [3, 4] # List: merge
```

6.2 Deep dive — the same interface, many behaviors

- Polymorphism = same name, many behaviors.

- Overriding: the child's version wins at runtime.
- Overloading: only one definition survives in Python — emulate it with defaults.
- Operator overloading: `+` on `str` concatenates, on `int` adds, on `list` merges — the class decides.
- The caller need not know the concrete type — **duck typing** — the object "explains itself".

Run the cells: `s.area(2)` gives a circle area, `s.area(3,4)` gives a rectangle area, and `'hello' + 'world'`, `4 + 5`, `[1,2,3] + [4,5]` each behave per their type.

PYTHON

```
class Shape:

    def area(self, a, b=0):
        if b == 0:
            return 3.14*a*a
        else:
            return a*b

s = Shape()

print(s.area(2))
print(s.area(3,4))
```

PYTHON

```
'hello' + 'world'
```

7. Abstraction

7.1 Contracts that force implementation

Abstraction hides the *how* and fixes the *what*. In OOP it is implemented with abstract base classes (`ABC`):

- `@abstractmethod` marks a method children are forced to write.
- An `ABC` cannot be instantiated directly — `BankApp()` raises `TypeError`.
- A child stays abstract until it implements every abstract method.
- Concrete methods are inherited and shared normally.

IDEA	EFFECT
Abstract class	a template/contract; cannot create objects of it
Abstract method	the child must provide an implementation
Concrete method	inherited as-is

Benefits: the parent *guarantees* an interface — consumers know `security()` and `display()` exist — while every child supplies its own details.

PYTHON

```

from abc import ABC, abstractmethod

class BankApp(ABC):
    def database(self):          # concrete – inherited and shared
        print('connected')
    @abstractmethod
    def security(self):          # abstract – child MUST implement
        pass

class MobileApp(BankApp):
    def security(self):          # the required implementation
        print('secure login added')

```

Deep Dive : Abstraction (ABC)

- **Abstract class** — defines an interface/contract
- `@abstractmethod` — the child MUST implement it
- An abstract class cannot be instantiated directly
- The child stays abstract until all abstract methods are defined
- The remaining (non-abstract) methods are inherited and shared
- Design: policy defined, implementation left to the child
- Guarantees which methods exist — consumers stay safe
- vs Interface (Python has no interface keyword) — ABC is the way
- Real: frameworks, base services

PYTHON

```

from abc import ABC, abstractmethod

class BankApp(ABC):

    def database(self):
        print('connected to database')

    @abstractmethod
    def security(self):
        pass

    @abstractmethod
    def display(self):
        pass

```

PYTHON

```

from abc import ABC, abstractmethod

class BankApp(ABC):
    def database(self):
        print('connected')
    @abstractmethod
    def security(self):      # child ko FORCE karta define karna
        pass

class MobileApp(BankApp):
    def mobile_login(self):
        print('login')
    def security(self):      # required!
        print('secure login added')

mob = MobileApp()           # abstract child bhi abstract → error
obj = BankApp()             # ❌ cannot instantiate abstract class

```

Abstraction (ABC)

- `ABC` + `@abstractmethod` → the children are **compelled** to implement the method
- An object of the abstract class cannot be created directly
- Benefits: template design — the parent guarantees, the child provides the details

PYTHON

```

class MobileApp(BankApp):

    def mobile_login(self):
        print('login into mobile')

    def security(self):
        print('mobile security')

    def display(self):
        print('display')

```

PYTHON

```
mob = MobileApp()
```

OOP Project — NLPApp**1. The nlpccloud API****1.1 Connecting to a hosted model**

`nlpccloud` wraps REST calls to hosted NLP models behind simple Python methods:

- `Client(model_name, api_key)` returns an authenticated client object.

- The model name selects which model runs (`finetuned-gpt-neox-20b` , `distilbert-base-uncased-emotion` , `python-langdetect`).
- Calls cost quota and need the network — failures are usually network, quota or invalid key issues, not a bug in your code.
- Responses arrive as JSON (Python dicts) that you must parse.
- Create the client once and reuse it for every call.
- Keep the API key secret in production — never commit it to a repository.

The first cell connects and calls `client.entities(...)` for named-entity recognition, returning a dict whose `entities` value is a list of found entities with positions and types. The language of this API mirrors OOP nicely: the `client` object knows how to talk to a model, and calls like `client.entities(text, ...)` are simply methods on that object — the library is already object-oriented before any of your classes appear.

PYTHON

```
import nlpcloud
client = nlpcloud.Client("finetuned-gpt-neox-20b", "YOUR_API_KEY")
```

Deep Dive : NLPApp — external API setup

- **nlpcloud** — third-party Python library (an API for NLP models)
- `Client(model_name, api_key)` — authenticated client object
- Models: sentiment (`distilbert-base-uncased-emotion`), language detect (`python-langdetect`), text gen
- API key — your account secret (nothing works without it)
- Response: a JSON dict → must be parsed
- Calls cost money/quota — keep the key secret in production
- Structured as: statement object + method with text
- Create the client once, reuse it for many calls
- Errors: network, quota, auth — a try/except is required

PYTHON

```
import nlpcloud

client = nlpcloud.Client("finetuned-gpt-neox-20b", "2b58d7fb9af09e617ee525e78c7766b6d8c5bb61", gpu=True, 1
client.entities("""John Doe started learning Javascript when he was 15 years old. After a couple of years
```

PYTHON

```
import nlpcloud
client = nlpcloud.Client("finetuned-gpt-neox-20b", "YOUR_API_KEY")
```

2. Packages, Installation & API Tours

2.1 Installing with pip from the notebook

A Jupyter notebook lets you run shell commands by prefixing them with `!`:

- `!command` executes a shell command inside the notebook cell.
- `pip install packagename` downloads and installs a Python package.
- A package is a library of modules (`nlpccloud`, `emoji`).
- Install once — you do not reinstall on every run.
- Upgrades: `!pip install --upgrade nlpccloud`; check installed packages: `!pip list`.
- Internet is required during installation.

On Google Colab, a bare `pip install nlpccloud` line also works directly. After installing, the import + client cells connect to the API. The exploration cells fire real requests — entity recognition, sentiment analysis and language detection. Take the structure in even when responses vary with the network or key. Watch the order of operations: install first, import second, call third. If the import cell fails with `ModuleNotFoundError`, the installation did not finish — run the pip cell again and wait for `Successfully installed`.

2.2 Deep dive — a quick tour of three models

- `sentiment()` returns `scored_labels` — one entry per emotion with a probability score.
- `langdetection()` returns the detected language and its confidence.
- `entities()` returns identified entities inside the text — here it hunts for programming languages in a short developer bio.
- In the sample sentiment result, `joy` carries a score near `0.998` — the model is almost certain.

These raw API answers are the *data* your app will parse. Understanding the JSON shape is the key skill: the app's job is to read these dicts and turn them into simple, useful messages. Notice that sending the same paragraph to `sentiment()` twice yields nearly identical scores — the model is stable enough for a demo. What does vary is the size and order of the JSON, which is exactly why you must parse the answer instead of hard-coding it.

PYTHON

```
pip install nlpccloud
# https://nlpcCloud.com/home/playground/language-detection
```

PYTHON

```
import nlpccloud

client = nlpccloud.Client("distilbert-base-uncased-emotion", "2b58d7fb9af09e617ee525e78c7766b6d8c5bb61", gp
client.sentiment("Look what's just come on the market in #ValThorens! A recently renovated, charming 6 bed
```

3. The NLPApp Class

3.1 One class runs the whole app

The program is a single class whose constructor starts the menu:

- `__database` — a private dict mapping email → `[name, password]`. Encapsulation keeps user data inside the class.
- The constructor calls `__first_menu()` — creating the app IS launching the app.
- Each menu action is its own private method: `__register`, `__login`, `__sentiment_analysis`. Modularity = one responsibility per method.
- Methods call each other: after registering, control returns to `__first_menu()`; after a successful login, `__second_menu()` opens.

State persists across method calls because it lives on `self` — the user interacts with the same object that remembers the database. This is how real interactive applications are organized. Because every method is private, the outside world can only trigger the flow through the constructor — no stray calls can start the app in a broken state. That is encapsulation applied to behavior, not just to data.

PYTHON

```
class NLPApp:
    def __init__(self):
        self.__database = {} # private: user records
        self.__first_menu() # the app starts immediately
```

NLPApp class — OOP structure

- The first menu is called from `__init__` → the app starts as soon as the object is created
- `__database` is private — encapsulation of user data
- Methods with the `__` prefix → controlled access
- Each menu option has its own method — modular code

PYTHON

```
class NLPApp:
    def __init__(self):
        self.__database = {} # private: users ka data
        self.__first_menu() # constructor me hi menu dikhan

    def __first_menu(self): ...
    def __register(self, name, email, password): ...
    def run(self): ...
```

Deep Dive : NLPApp class — structure

- OOP structure of the project — a single class, methods: login/register/start
- **init** — state (users dict, client)
- Methods group functionality: each API action is a method
- Menu-driven: a while loop + input
- CLI app pattern (console user interface)
- Methods do: sentiment / textgen / language

- State persists between method calls
- Design: one responsibility per method
- This is the pattern real projects use (MVC-lite)

PYTHON

```
import nlpcloud

class NLPApp:

    def __init__(self):

        self.__database = {}
        self.__first_menu()

    def __first_menu(self):

        first_input = input("""
        Hi! how would you like to proceed?
        1. Not a member? Register
        2. Already a member? Login
        3. Galti se aa gaye? Exit
        """)

        if first_input == '1':
            self.__register()
        elif first_input == '2':
            self.__login()
        else:
            exit()

    def __second_menu(self):

        second_input = input("""
        # ...
        self.__second_menu()

obj = NLPApp()
```

PYTHON

```

class NLPApp:
    def __init__(self):
        self.data = {} # state
        self.client = None # lazy init

    def login(self):
        ...

    def register(self):
        ...

    def start(self):
        # menu loop

```

4. Parsing the API Response

4.1 From raw scores to a human answer

The sentiment API hands back nested JSON; your app must extract the *most probable* label:

Breaking it down:

- `d['scored_labels']` → a list of dicts, each with `label` and `score`.
- `enumerate(L)` → pairs of `(position, score)` ready for sorting.
- `sorted(..., key=lambda x: x[1], reverse=True)` → highest score first.
- `[0][0]` → take the top pair, then its position index.
- That index finds the label with the maximum probability.

The whole trick is "extract scores → order them → pick the top label". The app performs exactly this inside `__sentiment_analysis`. Reading nested JSON is a transferable skill: every modern API returns dicts and lists, and the pattern stays the same — navigate one level at a time, collect what you need, and let the data shape drive your decisions.

PYTHON

```

d = {'scored_labels': [
    {'label': 'sadness', 'score': 0.98},
    {'label': 'joy', 'score': 0.01},
]}

L = [i['score'] for i in d['scored_labels']]
index = sorted(list(enumerate(L)), key=lambda x: x[1], reverse=True)[0][0]
d['scored_labels'][index]['label'] # 'sadness'

```

Parsing the API response

- The API's `scored_labels` list — a score for each label
- `enumerate` + `sorted(..., reverse=True)` — the index of the highest score
- `lambda x: x[1]` — sorts by score

- Top label = the emotion/language with the highest probability

PYTHON

```
d = {'scored_labels': [
    {'label': 'sadness', 'score': 0.98},
    {'label': 'joy', 'score': 0.001},
]}

L = [i['score'] for i in d['scored_labels']]
index = sorted(list(enumerate(L)), key=lambda x: x[1],
               reverse=True)[0][0]
d['scored_labels'][index]['label']
```

Deep Dive : Parsing the API response

- The API returns a JSON dict — a nested structure
- `d['scored_labels']` → a list of score entries
- Each entry is a dict: `{'label': 'sadness', 'score': 0.98}`
- Extract the scores → find the max → get the label of the max
- `max(L)` + `L.index()` — the top prediction
- Alternative: `max(d['scored_labels'], key=lambda x: x['score'])`
- The data shape matters — understand the API docs structurally
- JSON→dict is directly usable (the json module parses it)
- Parsing pattern: extract → aggregate → decide

PYTHON

```
d = {'scored_labels': [{'label': 'sadness', 'score': 0.98093181848526}, {'label': 'joy', 'score': 0.001407}
```

PYTHON

```
d
```

5. The emoji Module

5.1 A friendly touch of fun

- `emoji` is a small ecosystem library that replaces `:code:` placeholders with real emoji glyphs.
- Install it first: `!pip install emoji`.
- It is optional polish — nicer-looking output for a console app.
- The terminal must support the glyphs for the emoji to render.

The last cells install `emoji`, then convert `:grinning_face:` into a grinning face, proving the happy path works before you wire it into NLPApp. Emojis are the smallest of the three integrations here: like `nlpccloud`, they are a pip-installed friend you barely notice once installed — good practice for the install → import → use

rhythm that every third-party library follows. That rhythm — install once, import in every notebook or module, then call — governs all three libraries in this project, and it is the single most transferable habit from it. When an import fails, the fix is reinstall or restart the kernel, not rewriting code.

PYTHON

```
import emoji
print(emoji.emojize('Python is :grinning_face:')) # Python is 😊
```

emoji module

- `emoji.emojize()` — turns codes like `:grinning_face:` into emojis
- For showing output in the UI — a bit of fun
- You must install it first with `!pip install emoji`
- Showing an emoji in the NLP app's result = a user-friendly touch

PYTHON

```
import emoji
print(emoji.emojize('Python is :grinning_face:'))
```

Deep Dive : emoji module

- emoji — a library that turns `:name:` text into emojis
- `emojize('text with :code:')` — replaces the placeholder
- Needs install: `!pip install emoji`
- Fun/UX — a friendly touch in the output
- Not core Python — an ecosystem package
- Alternative: unicode emoji directly in the string
- Printing emoji in the console needs font support
- Real use: notifications, fun features
- Which codes: `emojize(':name:', language='alias')`

PYTHON

```
import emoji
print(emoji.emojize('Python is :grinning_face:'))
```

PYTHON

```
import emoji
print(emoji.emojize('Python is :grinning_face:')) # Python is
```

File Handling, Serialization and Deserialization

1. File Input/Output in Python

1.1 Text Data vs Binary Data

Every file on your hard disk is ultimately a sequence of bytes. What differs between file types is how those bytes are interpreted.

- **Text data** — for example the string `'12345'` — is stored as a sequence of Unicode characters, with each character encoded into one or more bytes (usually UTF-8).
- **Binary data** — for example the number `12345` — is stored as the raw bytes of its binary equivalent, with no character interpretation at all.

That distinction produces the two file families you deal with:

KIND	EXAMPLES	CONTENT
Text files	.txt, .md, .csv, .json, source code	human-readable characters
Binary files	images, music, video, .exe, PDFs	machine-oriented bytes

When Python opens a file in **text mode** it decodes the bytes into `str` objects; when it opens a file in **binary mode** it hands you raw `bytes`. Confusing the two is one of the most common beginner errors — and it is exactly what the mode argument of `open()` controls. Throughout this session, keep asking yourself: *is the file I am touching text or bytes?*

1.2 The Three Steps of File I/O

Almost every programming language follows the same ritual when working with files:

1. **Open a file** — call `open(path, mode)` and get back a file object (your handle on the file).
2. **Read or write data** — push text or bytes through that handle.
3. **Close the file** — call `f.close()` when you are done.

Think of a file handle like a telephone line. Without `open()` there is nothing to talk to; without `close()` the line stays occupied and the operating system resource (a file descriptor) is left dangling. Step 3 is easy to forget but it matters: an unclosed file can stay half-written on disk, stay locked against other programs, or leak memory, especially in long-running scripts. The memory buffer of an unclosed file may never reach the disk.

The mode string you pass to `open()` decides what step 2 is allowed to do. The next several cells dig into the write side first — starting with the most common (and most dangerous) writing mode, `'w'`.

2. Writing To Files

2.1 `open()` and the `write()` Method

The gateway to any file is the built-in `open()` function. You call it with the file path and a mode string, and it returns a file object — your connection to the file. In this section we write with `'w'` (write mode):

- If the file does **not** exist yet, `'w'` creates a brand-new empty file.
- If the file **already exists**, `'w'` truncates it immediately — every byte of the old content is erased before you write a single character.

Writing is done with `f.write(text)`, which places the given characters at the current pointer position. Two facts new learners miss:

- `write()` does **not** add newlines. `'hello world'` is written exactly as given; if you want a line break, you must include `'\n'` yourself.
- You must close the file when finished. `f.close()` flushes whatever is sitting in the memory buffer out to the disk and releases the system resource.

The first code cell demonstrates the creation behaviour: `'Hello world'` is written into a fresh `sample.txt`, and then a stray `f.write('hello')` after closing fails with `ValueError` — proof that the handle is truly dead. The second cell writes two strings back-to-back, showing that consecutive `write()` calls simply append characters in order.

open() modes — “w” write

- `open(path, mode)` — gives a file handle; `'w'` = write mode
- If the file does **not exist**, a new one is created; if it **exists**, the old content is erased
- `f.close()` is necessary after writing — frees resources
- `f.write('...')` — returns a string (how many chars were written)

PYTHON

```
f = open('sample.txt', 'w')
f.write('Hello world')
f.close()
```

Deep Dive : Writing multiline strings

- Each `f.write()` call = one continuous write
- File lines: newlines **you** have to add ()
- If there is no `\n` in the `write()` string, the lines get concatenated!
- `\n` = newline character (universal, 1 char)
- Windows/types: `\r\n` vs `\n` — Python text mode handles this
- More content: `writelines(L)` (a list of lines)
- Security: `write` overwrites the existing content completely
- Always close, or use the `with` block
- Text encodings matter: unicode emoji needs `utf-8`

PYTHON

```
f = open('sample1.txt', 'w')
f.write('hello
')
f.write('hi
')
f.write('how are you
')
f.close()
```

Deep Dive : open() with 'w' mode — write

- `open(path, mode)` returns a **file object**
- **'w' — write mode:**
 - if the file doesn't exist → CREATE it
 - if the file exists → TRUNCATE (overwrite!)
- `f.write(text)` — writes the string to the file
- **f.close()** — CRITICAL: flushes the buffer, frees resources
- Forgetting close → risk of data loss / locked memory
- `with open(...)` — auto-close (context manager)
- Handle `\n` correctly — `f.write('hello \n')` adds the newline manually
- write only strings (later: numbers to str)
- Encoding defaults to utf-8
- write returns the count of chars written

PYTHON

```
# case 1 - if the file is not present
f = open('sample.txt', 'w')
f.write('Hello world')
f.close()
# since file is closed hence this will not work
f.write('hello')
```

PYTHON

```
# write multiline strings
f = open('sample1.txt', 'w')
f.write('hello world')
f.write('\nhow are you?')
f.close()
```

2.2 The Dangers of 'w' — It Wipes Everything

Open a file with `'w'` and the old contents are gone before the first write. In the code below, `sample.txt` previously held `'Hello world'`; opening it with `'w'` and calling `f.write('salman khan')` replaced the entire file — and there is no undo button.

This truncate-then-write behaviour is the classic file-handling bug: you open a file with `'w'` intending to add a line, and silently destroy the previous data. Here is the mode decision table to memorise:

MODE	EFFECT ON EXISTING CONTENT
<code>'w'</code>	Truncates (erases) everything, then writes
<code>'a'</code>	Keeps everything, appends new data at the end
<code>'r+'</code>	Keeps everything, allows reading and writing
<code>'x'</code>	Exclusive create; fails if the file already exists

Rule of thumb: never open a file with `'w'` unless you are certain the old data is disposable. When you only need to add to a file, reach for `'a'` first.

Choosing The Right Mode

MODE	BEHAVIOUR
<code>'r'</code>	read only (the default)
<code>'w'</code>	write; truncates existing content
<code>'a'</code>	append; keeps existing content
<code>'x'</code>	exclusive create; fails if the file exists

Pick `'a'` when preserving old data matters and keep `'w'` for fresh rewrites. This single habit prevents most accidental data-loss disasters.

PYTHON

```
# case 2 - if the file is already present
f = open('sample.txt', 'w')
f.write('salman khan')
f.close()
```

2.3 Append Mode — 'a'

Append mode answers the question: how do we add to a file without destroying what is already there?

Opening with `'a'` keeps all existing content and places the file pointer at the very end, so every `write()` adds new characters after the last line.

Notice the difference from `'w'`:

- `'w'` empties the file the moment it is opened.
- `'a'` never removes content; it only extends the file.

Append also creates the file if it does not exist, which makes it a safe default for logs, counters and any scenario where a history of writes must be preserved. In the example, `'\nI am fine'` is appended to `sample1.txt` — the leading newline guarantees the new text starts on its own line instead of being glued to the previous last line. And because the pointer sits at the end, a fresh `open(..., 'a')` followed by another write will never overwrite anything you appended before.

Dangers of 'w' — append mode

- If the file contains "Hello World", opening with `'w'` wipes it immediately
- `'a'` (append) — adds content **at the end** of the file, never overwrites above
- Accidentally opening twice in `'w'` mode → the data will be gone

PYTHON

```
f = open('sample1.txt', 'a') # 'a' = append
f.write('
I am fine') # appended at the end
f.close()
```

Deep Dive — What Wiping The File Means

Truncation is not 'delete then write': the moment `open('sample.txt', 'w')` executes, the file's length is cut to zero, and the write that follows lands in an empty file. This is an operating-system-level operation, not a Python quirk — `open()` makes a system call that acquires a file descriptor, and Python layers buffering on top of it. The buffer accumulates writes in memory and flushes to disk when the file is closed (or on an explicit `flush()`, or when a context manager exits), which is why an unclosed file can lose data.

The mode letters break down as `r` (read), `w` (write, truncate), `a` (append), `x` (exclusive create), with `+` variants (`r+`, `w+`, `a+`) for read-and-write combos, and a `t` / `b` suffix to choose text or binary. If the old content matters, back it up first or pick append.

PYTHON

```
# Problem with w mode
# introducing append mode
f = open('/content/sample1.txt', 'a')
f.write('\nI am fine')
f.close()
```

3. Writing Multiple Lines — writelines()

3.1 write() vs writelines()

`f.write()` accepts a single string — perfect for small pieces of text. But when you already have a list of lines, looping over `write()` calls feels clumsy. Python provides `f.writelines(L)`, which takes an iterable of strings and writes each element in order.

Two details matter:

- `writelines()` does **not** add newlines for you. If the list is `['hello', 'hi']`, the output is the single line `hellohi` unless each element carries its own `'\n'`. In the demo, the list stores `'hello\n'`, `'hi\n'`, `'how are you\n'` and `'I am fine'` (no trailing newline), so the file comes out as four lines exactly as asked.
- Every element must be a string. Pass an integer or float and Python raises `TypeError`.

Three more practical notes. First, the target folder must already exist — opening a file inside a missing directory raises `FileNotFoundError`. Second, the output order always matches the list order. Third, for many lines, `writelines()` is usually faster than a Python loop of individual `write()` calls, because the heavy lifting is delegated to a single C-level operation.

writelines()

- `f.write(s)` — writes one string
- `f.writelines(L)` — writes a list of strings all at once
- You have to put `'\n'` in every element yourself — `writelines` does not add newlines automatically
- The folder path `/content/temp/` should exist beforehand, otherwise `FileNotFoundError`

PYTHON

```
L = ['hello
', 'hi
', 'how are you
', 'I am fine']
f = open('temp/sample.txt', 'w')
f.writelines(L)  # List of strings ek saath likhe
f.close()
```

Deep Dive — Lines Are Your Responsibility

A common early mistake is expecting `writelines()` to add `'\n'` itself. It does not: every list element is copied verbatim, so two elements without trailing newlines merge into one line. That is why the demo stores `'I am fine'` *without* a trailing newline while its neighbours keep theirs — the file ends exactly as intended.

Elements must all be strings; any other type triggers `TypeError`. And if a call ever runs away, remember the order of the output is simply the order of the list. For genuinely many lines, `writelines()` beats a loop of `write()` calls because the per-call Python overhead disappears.

PYTHON

```
# write lines
L = ['hello\n', 'hi\n', 'how are you\n', 'I am fine']

f = open('/content/temp/sample.txt', 'w')
f.writelines(L)
f.close()
```

4. Reading From Files

4.1 The read() Method

Reading mirrors writing. You open the file with `'r'` (read mode — also the default when the mode argument is omitted) and then pull text out through the file object. The simplest tool is `f.read()`:

- `f.read()` returns the whole remaining file content as a single string.
- `f.read(n)` returns only the next `n` characters (fewer if the end of the file is reached first).

Two behaviours are worth internalising right away. First, reading advances the file pointer: after an un-argued `f.read()`, the pointer rests at the end, so a second `f.read()` returns an empty string. Second, if the requested file does not exist, `open()` raises `FileNotFoundError` — Python refuses to invent content.

In the code cells, `sample.txt` was written earlier with four lines, so `read()` reproduces exactly:

and `read(10)` stops after ten characters (`hello\nhi\nh`). Because `read()` pulls the entire file into memory at once, it is a good idea only for small-to-medium files; for huge logs we prefer chunked or line-based reading, which is the subject of the next cells.

PYTHON

```
hello
hi
how are you
I am fine
```

read()

- `'r'` = read mode (this is the default too)
- `f.read()` — the entire file content as one string
- `f.read(n)` — only the first `n` characters
- Opening with `'r'` is required for reading — you can't read in `'w'` mode

PYTHON

```
f = open('sample.txt', 'r')
s = f.read()      # whole file string me
print(s)
f.close()
```

Deep Dive — Pointer Behaviour and Memory

`read()` returns everything from the current pointer position to the end — nothing more; once the pointer reaches the end, further `read()` calls return `''`. If the path is wrong, `open()` raises `FileNotFoundError` before any reading begins. Because `read()` materialises the whole file as one string, it is the wrong tool for enormous files: the entire content occupies memory at once.

For small configs and teaching examples it is perfect; for big files switch to `read(n)` chunks or line iteration. The returned text usually contains `'\n'`; call `.strip()` before parsing. `readlines()` — the sibling that returns a list of lines — trades the same full-file memory cost for easier line access.

PYTHON

```
# reading from files
# -> using read()
f = open('/content/sample.txt','r')
s = f.read()
print(s)
f.close()
```

PYTHON

```
# reading upto n chars
f = open('/content/sample.txt','r')
s = f.read(10)
print(s)
f.close()
```

4.2 readline() vs readlines()

When a file is large, or when you want to process it line by line, `read()` can be too greedy. Python offers two gentler tools:

- `f.readline()` — reads exactly one line (including its trailing newline) and returns it as a string. Each call advances the pointer by one line.
- `f.readlines()` — reads every remaining line at once and returns a list of strings.

The classic idiom for walking a whole file line-by-line is:

Loop until `readline()` returns an empty string (the file is exhausted), then stop. Note the `end=''` argument: `readline()` already includes `'\n'`, so printing without the extra newline keeps the output clean. The code cells show `readline()` called twice (printing `hello` and `hi` only) and then the while loop walking through all four lines. For modern code the even simpler `for line in f:` — iterating over the file object itself — gives the same result with less ceremony, and `readlines()` is best only when you truly need the whole list in memory at once.

PYTHON

```
while True:
    data = f.readline()
    if data == '':
        break
    else:
        print(data, end='')

```

Reading Line By Line

`f.readline()` consumes one line at a time, newline included. Because the pointer advances, a second call returns the next line. A `while` loop that reads until `readline()` returns `''` walks the entire file using almost no memory. The same job can be done more idiomatically with `for line in f:`, which iterates over the file object itself.

Deep Dive : readline() vs readlines()

- `readline()` — a single line per call, returns ""
 - loop: `for line in f:` — the file itself is iterable! (simplest)
- `readlines()` — a list of all lines (full load)
- Iteration idiom: `for line in f: print(line, end='')`
- Line endings: the content will contain `\n`
- Memory: line-by-line is cheap; `readlines` is expensive for big files
- The file pointer advances with every read
- Cleaner: `strip()` the lines for parsing

PYTHON

```
# readline() -> to read line by line
f = open('/content/sample.txt', 'r')
print(f.readline(), end='')
print(f.readline(), end='')
f.close()
```

PYTHON

```
# reading entire using readline
f = open('/content/sample.txt', 'r')

while True:

    data = f.readline()

    if data == '':
        break
    else:
        print(data, end='')

f.close()
```

5. Context Managers — the with Statement

5.1 Automatic Resource Cleanup

Calling `f.close()` manually works, but it is easy to forget — and if an exception jumps out of the middle of a block, the `close()` line may never run. Python's answer is the **context manager**: the `with` statement guarantees cleanup when the block ends, whether it ends normally or with an error.

When the indented block finishes, Python automatically calls `close()` on the file object — no explicit `close()` required, no resource leak, and cleanup happens even if an exception is raised inside. That is exactly why the next cell fails: after the `with` block, the handle `f` is closed, and `f.write('hello')` raises `ValueError` about operating on a closed file.

Under the hood `with` relies on two magic methods, `__enter__()` (runs at the start and returns the object bound to `as`) and `__exit__()` (runs at the end and performs cleanup). Files, sockets, database connections and locks all implement this protocol, which is why `with` is the modern, idiomatic way to handle resources in Python. The third cell re-opens `sample.txt` for reading inside a `with` block and confirms `readline()` still works reliably.

SQL

```
with open('sample.txt', 'w') as f:
    f.write('selmon bhai')
```

Deep Dive — Why with Always Cleans Up

`with open(path, 'w') as f:` triggers `__enter__()` to obtain and return the file object; the block runs; then `__exit__()` closes the handle no matter how the block exits — normally, via `return`, or via an exception. That guarantees the file is never left dangling.

The failed `f.write('hello')` *after* the block proves the cleanup really happened: the file object raises `ValueError` because its underlying resource is gone. This same protocol is why file objects, sockets, locks and database cursors all support `with` — one idiom, universally safe. You can also stack managers on one line: `with open(a) as f, open(b) as w:`, and modern Python (3.10+) supplements it with explicit `with (...) as (...)` grouping.

SQL

```
# with
with open('/content/sample1.txt', 'w') as f:
    f.write('selmon bhai')
```

SQL

```
f.write('hello')
```

6. The File Pointer — seek() and tell()

6.1 Reading In Chunks

Every open file has an invisible **pointer** that records the byte position of the next read or write. Two methods let you ask where the pointer is and move it:

- `f.tell()` — returns the pointer's current position (in bytes).
- `f.seek(n)` — moves the pointer to byte position `n`.

When you open a file and start reading, the pointer sits at 0 and creeps forward with every read. That is why a plain sequence of `f.read(10)` calls lines up consecutive ten-character chunks until the file ends. **Chunked reading** is the memory-friendly way to process big files: instead of loading a 1 GB log into RAM, you read it in small pieces, process each piece and discard it.

The notebook demonstrates this with `big.txt` — one thousand copies of `'hello world '` written by `writelines()` — then reads it back ten characters at a time. The output shows chunks like `d hello wo***o world he***` because every chunk is ten characters and the demo joins them with `'***'`. Finally, the seek-and-tell cell jumps straight to byte 15 with `f.seek(15)`, reads ten characters, and `tell()` reports 25 — the pointer really did advance by ten.

Tracking The Pointer

Every read or write nudges the file pointer forward. `tell()` reports the current byte offset; `seek(n)` jumps to an absolute offset from the start of the file. In the demo, reading ten characters and then calling `tell()` shows the offset growing by ten each time — the pointer state is entirely under your control.

Deep Dive : seek() & tell()

- **tell()** — the file pointer's position
- **seek(offset, whence)** — moves the pointer
 - whence=0 start, 1 current, 2 end
- Reading tip: read the file chunk by chunk (memory friendly):
- Seek works during write too — overwrite at any position
- Byte based for binary; char based for text
- Real: chunked processing of big logs/files
- `f.seek(0)` — rewind to the start

SQL

```
# moving within a file -> 10 char then 10 char
with open('sample.txt','r') as f:
    print(f.read(10))
    print(f.read(10))
    print(f.read(10))
    print(f.read(10))
```

SQL

```
# benefit? -> to load a big file in memory
big_L = ['hello world ' for i in range(1000)]

with open('big.txt','w') as f:
    f.writelines(big_L)
```

6.2 seek() While Writing

`seek()` is not limited to reading. In write mode the pointer moves exactly the same way, which turns any file into something you can patch. The code cell writes `'Hello'`, then calls `f.seek(0)` to rewind to the start, then writes `'Xa'`. The result: `'Xa'` overwrites the first two characters of `'Hello'` and the file ends up as `'Xallo'`.

This offset-based editing is how you replace or update a specific part of a file without rewriting the whole thing — handy for binary formats that store headers, and for checkpoint files where only a small footer changes. By default `seek(n)` counts bytes from the start; pass a second argument (whence) to anchor instead at the current position (1) or the end of the file (2).

Patching Files With `seek()`

Writing also moves the pointer, so `seek()` lets you overwrite a specific region. After `'Hello'`, `f.seek(0)` rewinds to the start and `f.write('Xa')` replaces the first two characters, yielding `'Xa11o'`. With the optional whence argument you can move relative to the start, the current position, or the end of the file.

SQL

```
# seek during write
with open('sample.txt', 'w') as f:
    f.write('Hello')
    f.seek(0)
    f.write('Xa')
```

7. Working With Binary Files

7.1 Text Mode vs Binary Mode

Everything so far assumed textual data, but many files — images, music, videos, compressed archives, PDFs — are binary. Every file is a bag of bytes; text mode simply means "decode those bytes as text". Tell `open()` that a `.png` is a text file and it will try to decode image bytes as UTF-8 characters, throwing unicode decode errors almost immediately.

The fix is to match the mode with the content:

- `'rb'` — read bytes (binary read). `f.read()` returns a `bytes` object, not a `str`.
- `'wb'` — write bytes (binary write). `f.write()` expects a `bytes` object.

Copying a binary file is therefore a pure byte-shuffle: open the source with `'rb'`, open a destination with `'wb'`, and copy via `f.read()` then `wf.write(data)` — all at once for small files, or chunk by chunk for large ones. The first code cell shows opening `screenshot1.png` in text mode failing; the second shows the correct byte-level copy producing an exact duplicate, `screenshot_copy.png`.

Binary mode also skips newline translation: text mode converts platform line endings automatically, while binary mode hands you the raw bytes verbatim — exactly right when the content is not text at all.

Copy binary file

- `'rb'` / `'wb'` — binary read/write (images, zip files)
- Text mode fails on images (encoding error)
- Small files: `f.read()` + `wf.write(...)` together
- Big files: copy chunk by chunk (in the code later)

SQL

```
with open('screenshot1.png', 'rb') as f:
    with open('screenshot_copy.png', 'wb') as wf:
        wf.write(f.read())
```

Deep Dive : Binary files — 'rb' mode

- Text mode expects text (utf-8 decode) — images fail
- `'rb'` — reads bytes; `'wb'` — writes bytes
- Binary data: a bytes object (non-text)
- Copy files: read bytes → write bytes
- Big binaries: chunked copy (efficient)
- Binary files: images, PDFs, audio, pickle
- text mode does newline conversion — binary is raw
- `decode('utf-8')/encode`: byte ⇌ string
- Always: make the mode match the content type

SQL

```
# working with binary file
with open('screenshot1.png', 'r') as f:
    f.read()
```

SQL

```
# working with binary file
with open('screenshot1.png', 'rb') as f:
    with open('screenshot_copy.png', 'wb') as wf:
        wf.write(f.read())
```

8. Text Files Only Store Strings

8.1 The TypeError Problem

Text files have a hard rule that surprises beginners: **you can only ever write strings**. Everything else — integers, floats, booleans, lists, dicts — must first be converted.

That is exactly why the code cell with `f.write(5)` raises `TypeError`. Integers live in memory as machine values; a text file stores characters. To save the number 5 you must write `'5'`, and the round trip requires parsing on the way back:

The same rule applies to complex structures: `str(d)` produces a text *representation* of a dict, but trying to reparse that text with `dict(f.read())` fails because the stringified form is not valid Python syntax. Plain text files simply cannot round-trip arbitrary Python objects. That limitation motivates the second half of this session: serialization libraries — JSON and pickle — encode objects into a form that can be stored and reliably decoded.

SQL

```
with open('sample.txt', 'w') as f:
    f.write(5)          # TypeError: write() argument must be str
```

SQL

```
with open('sample.txt', 'r') as f:
    print(int(f.read()) + 5)  # reads '5', converts to int, adds 5 → 10
```

String Conversion Round Trip

Text files cannot store raw numbers — `f.write(5)` raises `TypeError`. Convert with `str()` before writing and parse with `int()` or `float()` after reading. For structured data (dicts, lists), plain `str()` produces text you cannot safely parse back; use JSON or pickle instead.

Deep Dive : Only strings in text files

- In text files you can write ONLY strings
- int/float/dict → first do **str()** or serialize
- Complex data (dict/list) into binary/json:
- Proper way: **json / pickle** (later)
- Reading with a type cast: `int(f.read())`, `float()`
- Conversions are manual — serialization modules come later
- str vs repr for data round-trip

SQL

```
# working with other data types
with open('sample.txt', 'w') as f:
    f.write(5)
```

SQL

```
with open('sample.txt', 'w') as f:
    f.write('5')
```

Serialization and Deserialization

- **Serialization** - process of converting python data types to JSON format
- **Deserialization** - process of converting JSON to python data types

What is JSON?

```

1 {
2   "d": {
3     "results": [
4       {
5         "__metadata": {
6           "type": "EmployeeDetails.Employee"
7         },
8         "UserID": "E12012",
9         "RoleCode": "35"
10      }
11    ]
12  }
13 }

```

JSON — Python ↔ File

- Serialization = converting Python data into a **text/JSON** string
- Deserialization = JSON back to a Python object (the reverse)
- `json.dump(obj, file)` — write; `json.load(file)` — read
- dict keys, nested dicts, tuples all handled

JSON serialization is the art of turning Python data into plain text that any system can read.

- **Serialization** — Python object → JSON text (on disk or in a string).
- **Deserialization** — JSON text → Python object.
- `json.dump(obj, file)` writes to an open file; `json.load(file)` reads it back.
- dict keys become strings, tuples become arrays (lists), booleans and `None` map cleanly.

Nested dicts, lists inside dicts — the encoder handles all of these recursively, which is why a single `json.dump()` call can save an entire data structure in one shot.

SQL

```

import json

L = [1, 2, 3, 4]
with open('demo.json', 'w') as f:
    json.dump(L, f)      # write karo JSON format me

with open('demo.json', 'r') as f:
    d = json.load(f)     # wapas Python object me

```

Deep Dive : JSON — serialization

- **Serialization** — Python object → storable/transferrable format
- **Deserialization** — format → Python object
- `json.dump(obj, file)` — straight to a file
- `json.dumps(obj)` — to a STRING

- `json.load(file)` / `json.loads(str)` — back
- JSON is universal (web, APIs) — language-agnostic
- dict keys→str; tuple→array (becomes list)
- Nested dicts fine; custom classes NOT (need an encoder)
- indent: `json.dump(obj, f, indent=4)` — pretty print
- Human-readable data transfer standard

SQL

```
import json
L = [1, 2, 3, 4, 5]
with open('demo.json', 'w') as f:
    json.dump(L, f)      # Python → file (JSON text)

with open('demo.json', 'r') as f:
    data = json.load(f)  # file → Python
```

9. Serialization and Deserialization — JSON

9.1 dump(), dumps(), load() and loads()

Serialization means converting a Python object into a format that can be written to a file or sent over a network; **deserialization** is the reverse — turning that stored text back into a live Python object.

JSON (JavaScript Object Notation) is the world's favourite interchange format: a plain-text, language-agnostic way to represent data that browsers, servers and configuration tools all understand. Python's `json` module maps JSON types onto Python types:

JSON	PYTHON
object	dict
array	list
string	str
number	int / float
true / false / null	True / False / None

The `json` module offers four functions:

- `json.dump(obj, file)` — serialize `obj` into an open text file.
- `json.dumps(obj)` — serialize `obj` into a `str`.
- `json.load(file)` — read JSON from an open file and deserialize it.
- `json.loads(text)` — deserialize from a `str`.

The code cells exercise the whole pipeline: a list serialized into `demo.json`, a dict written with `indent=4` for pretty output, a read-back that prints the original dict, a tuple (which comes back as a list), and a nested structure — a dict holding a list of marks — all handled automatically. In short: `json` covers dict, list, str, int, float, bool and `None` for free; anything fancier needs your help, which is the next section's topic.

SQL

```
# serialization using json module
# list
import json

L = [1,2,3,4]

with open('demo.json','w') as f:
    json.dump(L,f)
```

SQL

```
# dict
d = {
    'name': 'nitish',
    'age': 33,
    'gender': 'male'
}

with open('demo.json','w') as f:
    json.dump(d,f,indent=4)
```

10. Serializing Custom Objects With json

By default `json` only understands the basic types from the table on the previous page. Hand it a `Person` instance and it raises `TypeError` — there is no obvious textual form for an object. This section teaches `json` to convert your own classes.

Serializing custom objects (str return)

- `json` does not understand custom classes by default → error
- The `default=` argument — a function that turns the object into a string/dict
- `isinstance(obj, Person)` — type check inside the converter
- `json.dumps()` vs `json.dump()` — the latter writes to a file

PYTHON

```
class Person:
    def __init__(self, fname, lname, age, gender):
        self.fname = fname

import json

def show_object(person):
    if isinstance(person, Person):
        return f"{person.fname} {person.lname} age -> {person.age} ..."

json.dumps(person, default=show_object)
```

Deep Dive : Custom object serialization

- json ONLY handles: dict, list, str, int, float, bool, None
- A custom class → TypeError by default
- Solutions:
 1. a `show_object` function (str) — convert to str/dict first
 2. a dict conversion helper:
- Deserialization is the reverse: read the dict → construct the object
- Generic: `default=dump_fn` — json handles unknown types via the callback
- Alternative: the **dict** trick (instance attrs)
- Custom protocols = encode/decode pairs
- Real: data classes + `dataclasses.asdict`

PYTHON

```
class Person:
    def __init__(self, fname, lname, age, gender):
        self.fname = fname; self.lname = lname
        self.age = age; self.gender = gender

person = Person('Nitish', 'Singh', 33, 'male')
```

PYTHON

```
def show_object(p):
    if isinstance(p, Person):
        return {'fname': p.fname, 'lname': p.lname, 'age': p.age, 'gender': p.gender}

json.dump(person, f, default=show_object)
```

10.1 json's Built-in Type Table

A custom class like `Person` has no built-in JSON representation. When you try `json.dump(person, f)` directly, the encoder inspects the object's attributes, finds no standard mapping, and gives up with `TypeError: Object of type Person is not JSON serializable`.

The reason is that JSON has no notion of a *class*. It can express dicts, lists, strings and numbers, but nothing about methods or identity. Converting an object to JSON therefore means choosing a **representation** — typically either:

1. A formatted string that describes the object, e.g. `'Nitish Singh age -> 33 gender -> male'`.
2. A plain dict of its attributes, e.g. `{'name': 'Nitish Singh', 'age': 33, 'gender': 'male'}`.

Both approaches appear in the code. The `Person` class below stores four attributes (`fname`, `lname`, `age`, `gender`), and an instance is created by passing four arguments (`Person('Nitish', 'Singh', 33, 'male')`). Which representation to choose is up to you — but whatever you pick, the same data must be recoverable when the JSON is read back.

PYTHON

```
class Person:

    def __init__(self, fname, lname, age, gender):
        self.fname = fname
        self.lname = lname
        self.age = age
        self.gender = gender

    # format to printed in
    # -> Nitish Singh age -> 33 gender -> male
```

PYTHON

```
person = Person('Nitish', 'Singh', 33, 'male')
```

10.2 The default= Converter — str vs dict

To tell `json` how to handle your object, pass a converter function through the `default=` parameter of `json.dump()` (or `json.dumps()`). Whenever the encoder meets a type it does not understand, it calls your function with that object and writes the returned value in its place.

The first code cell writes a **string** representation using `.format()`:

The second returns a **dict** instead:

Which is better? A string is readable but loses structure — you cannot pull out `person['age']` later, and turning it back into a `Person` is awkward. A dict preserves every attribute as a named key, mirrors standard JSON data, and makes deserialization trivial: `json.load()` returns the dict, and you simply pass those keys into the `Person` constructor. The `isinstance()` check keeps the converter safe — it only handles `Person` objects and leaves everything else to the default behaviour. The final cell of the section reads the saved dict back and prints it together with its type.

SQL

```
# As a string
import json

def show_object(person):
    if isinstance(person, Person):
        return "{} {} age -> {} gender -> {}".format(person.fname, person.lname, person.age, person.gender)

with open('demo.json', 'w') as f:
    json.dump(person, f, default=show_object)
```

SQL

```
# As a dict
import json

def show_object(person):
    if isinstance(person, Person):
        return {'name': person.fname + ' ' + person.lname, 'age': person.age, 'gender': person.gender}

with open('demo.json', 'w') as f:
    json.dump(person, f, default=show_object, indent=4)
```

indent parameter

- `indent=4` — pretty, readable, multiline output
- Without `indent` — compact one-line JSON
- Golden for readability in debugging and logs
- Best combo with the dict format converter

PYTHON

```
json.dump(obj, f, indent=4)
```

10.3 The indent Parameter — Pretty Output

By default `json.dump()` writes its output as one long, dense line — correct and parseable, but painful to read. Adding `indent=4` lays the structure out across multiple lines:

Every nested level gets four extra spaces of indentation. This setting is what you want for debug files, logs and configuration files a human will also read. Combine `indent` with the dict-returning converter for the most readable result, and remember the compact default is still your friend when file size or bandwidth matters most.

SQL

```
# deserializing
import json

with open('demo.json','r') as f:
    d = json.load(f)
    print(d)
    print(type(d))
```

SQL

```
json.dump(person, f, default=show_object, indent=4)
```

11. Pickling — Python's Binary Serialization

11.1 How Pickling Works

JSON shines when the data must travel between languages or live in readable text. But for Python-to-Python storage there is a faster, more faithful route: **pickle**. Pickling is the process whereby a Python object hierarchy is converted into a byte stream; unpickling is the inverse operation, whereby a byte stream (from a binary file or a bytes-like object) is converted back into an object hierarchy.

Because pickle stores the object's *type information together with its payload*, almost anything survives the round trip: custom classes, functions, generators, nested structures — none of which JSON can touch without a hand-written encoder. The price is a binary format: pickled data is not human-readable, and it is Python-specific, so a pickled file is useless to a program written in another language.

The code defines a friendly `Person` class (name + age) with a `display_info()` method and creates one instance. A key fact: the class definition must be importable at unpickling time, because the byte stream references the class by name rather than copying its code.

PYTHON

```
class Person:

    def __init__(self,name,age):
        self.name = name
        self.age = age

    def display_info(self):
        print('Hi my name is',self.name,'and I am ',self.age,'years old')
```

PYTHON

```
p = Person('nitish',33)
```

Pickling — binary object store

- **Pickle** = Python object hierarchy → byte stream (`wb`)
- `pickle.dump(obj, file)` / `pickle.load(file)`

- Data types and custom objects all survive (type info + payload)
- SECURITY WARNING: never load an untrusted pickle (code execution!)

SQL

```
import pickle

p = Person('nitish', 33)
with open('person.pkl', 'wb') as f:
    pickle.dump(p, f)      # object as byte stream

with open('person.pkl', 'rb') as f:
    p2 = pickle.load(f)    # wap as object
p2.display_info()
```

A Word On Security

Pickle is Python-speak: it can execute code reconstructed from the stream. Loading a maliciously crafted `.pkl` can run arbitrary code on your machine. Therefore **only unpickle data you fully trust** — your own saved files, never someone else's download.

`joblib.pickle` (used to save scikit-learn models) carries the same caveat. When in doubt, prefer JSON, which can only ever represent plain data and cannot smuggle executable code into your process.

Deep Dive : Pickle — binary object store

- **Pickle** — Python objects directly to/from BINARY
- Any Python object (custom classes, functions, complex data) — the verdict is best
- `pickle.dump(obj, file)` / `pickle.load(file)`
- Binary format — not human readable
- Fast & native for Python-to-Python
- Security warning: NEVER unpickle untrusted data (code execution!)
- JSON vs Pickle:

	JSON	PICKLE
format	text	binary
read by	any lang	Python only
custom obj	manual encoder	direct
security	safe	dangerous untrusted

- Use: cache, saving ML models (joblib/pickle), session state

SQL

```
import pickle
p = Person('nitish', 33)
with open('person.pkl', 'wb') as f:
    pickle.dump(p, f)    # serialize → binary file

with open('person.pkl', 'rb') as f:
    p2 = pickle.load(f)  # deserialize back
print(p2.name, p2.age)
```

11.2 pickle.dump() and pickle.load()

Using `pickle` mirrors `json`, with one crucial difference — the files are opened in **binary mode**: `'wb'` for writing, `'rb'` for reading.

`pickle.dump(obj, file)` serializes `obj` into the open binary file; `pickle.load(file)` reads the byte stream and rebuilds the object. Notice this is a **new** object living at its own memory address — it merely happens to carry the same attribute values — and the `Person` class must be available in the current namespace.

The code confirms the round trip by calling `p.display_info()` on the read-back object; it prints exactly as a freshly built instance: `Hi my name is nitish and I am 33 years old`. That is the complete lifecycle: class definition, instance creation, dump to disk, load back, method call again.

SQL

```
# pickle dump
import pickle
with open('person.pkl', 'wb') as f:
    pickle.dump(p, f)
```

SQL

```
# pickle load
import pickle
with open('person.pkl', 'rb') as f:
    p = pickle.load(f)

p.display_info()
```

JSON vs Pickle

FEATURE	JSON	PICKLE
Format	Text (readable)	Binary
Language	Cross-language	Only Python
Custom objects	via default=	Direct
Safety	Safe	Unsafe (untrusted)
Use-case	API, config	Internal store

- JSON — take it when you need security/readability
- Pickle — a fast internal Python store, only trusted data
- `indent`, `sort_keys` — JSON's formatting aids

Pickle Vs Json

- Pickle lets the user to store data in binary format. JSON lets the user store data in a human-readable text format.

Exception Handling in Python

1. Syntax Errors — Failing Before You Run

1.1 What Is A Syntax Error?

A syntax error means the program text does not follow the grammar of the Python language. It is detected by the interpreter while the code is being parsed — before a single statement executes.

Key facts to remember:

- The error is raised by the compiler/interpreter during compilation, not by your logic.
- The program never runs; the parser fails immediately.
- It is fully fixable: correct the offending line and the obstacle disappears.
- The message usually points at the exact line and leaves a caret (`^`) under the spot where the parser got confused.

Think of syntax as the road signs of a language. A program with a syntax error is like a sentence with a broken word — a human can sometimes guess the meaning, but Python will not. Grammar mistakes block everything else, so they always have the highest debugging priority.

Syntax Error — compile-time

- Happens at compile/interpreter time — the program doesn't run, it fails directly
- The Python interpreter caught a mistake in its grammar
- Fix: rewrite the line with correct grammar
- Syntax errors are debuggable — the error message tells you where

PYTHON

```
print 'hello world' # missing parentheses!
```

Deep Dive : Syntax errors — compile time

- **Syntax error** — at parse time; Python can't compile the code
- The program doesn't even RUN — the parse fails first
- Causes: missing colon/parens, typo keywords, bad indentation
- `IndentationError` — the block was set up wrong

- Fix: correct syntax per Python rules
- Highest priority — fix the syntax first
- Debug: the error line + caret pointer
- vs runtime exception — an error during execution
- Syntax is not a problem with the code's logic, just the language rules

PYTHON

```
# Examples of syntax error
print 'hello world'
```

PYTHON

```
print 'hello world'      # ✖ SyntaxError - missing ()
a = 5
if a==3                  # ✖ missing :
    print('hello')
iff a==3:                # ✖ typo keyword
```

1.2 Typical Sources of Syntax Errors

Most syntax errors come from a small set of careless mistakes:

- **Leftover or missing symbols** — forgetting a colon, or forgetting to close a bracket, parenthesis or quote.
- **Misspelled keywords** — typing `iff` instead of `if`, or `whiler` instead of `while`.
- **Incorrect indentation** — mixing tabs and spaces, or indenting a block inconsistently; Python uses indentation to mark blocks, so it is picky.
- **Empty blocks** — an `if`, `else`, loop, class or function whose body is completely empty (Python expects at least one statement, usually `pass`).

The next few code cells show these failures in action. Run each cell, read the error line, and fix the code to make it run.

1.4 Indentation Shapes The Block

Indentation is not a style preference in Python — it is the block structure. After any header line ending in `:`, the statements that belong to it must be indented consistently (usually four spaces).

- `if a == 3:` then a plain `print('hello')` at column zero is `IndentationError`.
- `print('hello')` indented by two spaces, and another line indented by four, makes Python raise `TabError` or a `SyntaxError` about inconsistent use of tabs and spaces.
- Choose spaces over tabs, and keep the same indent width inside every block.

The colon-plus-indentation pair is the single most common cause of syntax errors for beginners. Once you train your eye to look for a missing `:` and a consistent indent, you will fix these in seconds.

1.3 Debugging The Three Classics

The next three code cells are ready-made crashes. They are meant to be read, not feared:

- **Missing colon** — `if a==3` with no trailing `:` raises `SyntaxError: invalid syntax`. The colon announces that a suite (an indented block) follows.
- **Misspelled keyword** — `iff a==3:` is not `if`, so the parser rejects it as an unknown name at that position. Python keywords are an exact vocabulary: `if`, `elif`, `else`, `while`, `for`, `def`, `return` and so on.
- **Broken indentation** — `if a==3:` followed by a `print()` that is not indented triggers `IndentationError: expected an indented block`. Because Python has no braces, indentation is the grouping mechanism; a body must be indented one level deeper than the header.

When you see `SyntaxError` or `IndentationError`, your first instinct should be: what did I type wrong, and where does the caret point? The error position identifies the line; your eye identifies the fix.

PYTHON
<pre>a = 5 if a==3 print('hello')</pre>
PYTHON
<pre>a = 5 iff a==3: print('hello')</pre>

2. Exceptions — Runtime Failures

2.1 The Common Built-in Exceptions

A running program can fail for reasons that have nothing to do with grammar. These runtime failures are **exceptions**. Python defines a whole family of built-in ones, each with a name that describes its cause:

EXCEPTION	WHEN IT HAPPENS
<code>IndexError</code>	Accessing an invalid index, e.g. <code>L[100]</code> on a 3-element list
<code>ModuleNotFoundError</code>	<code>import</code> of a module that cannot be found
<code>KeyError</code>	A key that is not in the dictionary, e.g. <code>d['age']</code>
<code>TypeError</code>	An operation applied to an inappropriate type, e.g. <code>1 + 'a'</code>
<code>ValueError</code>	A function receives a value it cannot use, e.g. <code>int('a')</code>
<code>NameError</code>	A name that has never been defined, e.g. <code>print(k)</code>
<code>AttributeError</code>	Accessing an attribute/method the object does not have, e.g. <code>L.upper()</code>

Every exception stops normal execution and produces a **stack trace** — a report showing where the error occurred. The first diagnostic skill you should build is simply *reading* that message and connecting the exception name to its cause.

2.2 Reading The Traceback

When an exception escapes unhandled, Jupyter shows a traceback whose final line is the actual error (`IndexError: list index out of range`, `TypeError: unsupported operand ...`). Read it bottom-up: the last line names the exception and a human-readable reason; the lines above are the call chain that led to it.

A few rules of thumb:

- The exception **name** is a strong hint: `IndexError` is about indices, `KeyError` about dict keys, `TypeError` about types, `ValueError` about values.
- `ZeroDivisionError` appears whenever you divide by zero; `FileNotFoundError` whenever `open()` cannot find a path.
- Exceptions form a hierarchy rooted at `BaseException`, with `Exception` as the common ancestor of nearly everything you will catch.
- The traceback's bottom line is the truth; the top lines are context.

You are not expected to memorise the list — you are expected to recognise the pattern. Run the next eight cells one by one and let each crash teach you its exception.

PYTHON

```
# IndexError
# The IndexError is thrown when trying to access an item at an invalid index.
L = [1,2,3]
L[100]
```

PYTHON

```
# ModuleNotFoundError
# The ModuleNotFoundError is thrown when a module could not be found.
import mathi
math.floor(5.3)
```

3. Handling Exceptions — try / except

3.1 Why Handling Matters

Exceptions are raised by the Python runtime whenever something unforeseen happens during execution: memory pressure, division by zero (a logical error), a missing resource, a failed network call, a database error. Without handling, such an exception crashes the whole program and dumps an ugly error in the user's face.

Handling flips that: you place the risky operation inside a `try` block, and the fallback behaviour inside an `except` block. If nothing goes wrong, `except` is skipped; if an exception appears, the `try` block is abandoned at that point and the matching `except` handler takes over.

The user sees a friendly message and the program keeps running instead of dying. That is the entire motivation for this session: handle failures *where they can happen*, and let the user continue.

SQL

```
try:
    with open('sample1.txt', 'r') as f:
        print(f.read())
except:
    print('sorry file not found')
```

3.2 The try / except Pattern

`try` / `except` is Python's safety net. The mechanics are simple:

- The `try` block holds the code that *may* raise an exception.
- The `except` block holds the fallback that runs *if* an exception occurs.
- If the `try` block completes without errors, the `except` block is ignored entirely.
- If an exception occurs partway through `try`, control jumps immediately to `except` and the rest of the `try` body never runs.

A bare `except:` catches literally anything — convenient for demos, but a clumsy production habit because it swallows the error without telling you which exception it was. The common files are handled with *specific* `except` clauses (next topic). A warning: even `except Exception as e:` is a broad net; the idiomatic path is to catch the specific exceptions your code can actually react to.

3.3 What Happens Inside try

A note on how the net behaves:

- Once an exception is raised, the remainder of the `try` block is skipped — statements after the failing line never execute.
- The matching handler is the first `except` clause whose exception class matches, checked top to bottom.
- The code demo above opens `sample1.txt` for reading; that file does not exist, `open()` raises `FileNotFoundError`, the friendly message prints, and the notebook keeps running — no crash, no red traceback.

The same pattern is used everywhere real programs live: opening files, validating input, calling APIs, parsing data. Wrap risk, react gracefully, and let the rest of the program continue.

SQL

```
# Why is it important to handle exceptions
# how to handle exceptions
# -> Try except block
```

SQL

```
# let's create a file
with open('sample.txt', 'w') as f:
    f.write('hello world')
```

3.4 Catching Specific Exceptions

Catching every error with one blanket `except` is often too blunt — you usually want the *right* response per failure. Python lets you stack multiple handlers:

Rules that make this work:

- Handlers are checked **in order** against the raised exception class.
- Put the **most specific** exceptions first and the general catch-all (`except Exception as e:`) last, as the safety net.
- `as e` binds the exception object itself, so `print(e)` shows its message (`[Errno 2] No such file or directory: 'sample1.txt'`).

The demo cell raises several candidate errors on purpose: `open()` on a missing file, printing a name, dividing, indexing a list. Only the first matching handler runs — which is why you should keep the order specific → general.

PYTHON

```
# catching specific exception
try:
    m=5
    f = open('sample1.txt','r')
    print(f.read())
    print(m)
    print(5/2)
    L = [1,2,3]
    L[100]
except FileNotFoundError:
    print('file not found')
except NameError:
    print('variable not defined')
except ZeroDivisionError:
    print("can't divide by 0")
except Exception as e:
    print(e)
```

PYTHON

```
try:
    ...
except FileNotFoundError:
    print('file not found')
except NameError:
    print('variable not defined')
except ZeroDivisionError:
    print('...')
except Exception as e:
    print(e)
```

3.5 The else Clause

A third piece completes the picture: `else`. It appears after the `except` clauses and runs **only if the `try` block completed without any exception**.

If the file does not exist, the `except` path runs and `else` is skipped; if the file opens happily, `else` proceeds to read it. `else` is valuable because code that should only execute on success is kept visually separate from the risky section — no accidental exception swallowing, clearer intent.

PYTHON

```
# else
try:
    f = open('sample1.txt', 'r')
except FileNotFoundError:
    print('file nai mili')
except Exception:
    print('kuch to lafda hai')
else:
    print(f.read())
```

PYTHON

```
try:
    f = open('sample1.txt', 'r')
except FileNotFoundError:
    print('file not found')
else:
    print(f.read())
```

3.6 try / except / else / finally — The Complete Flow

The full pattern is `try → except → else → finally`:

- `try` — the risky code.
- `except` — runs when an exception occurred (one or many, in specificity order).
- `else` — runs only when `try` finished with no exception.
- `finally` — *always* runs, whether an exception happened or not, even if `try` or `except` used `return` or `raise`.

`finally` is the natural home for cleanup: close a file, release a lock, commit or roll back a database connection. Because the language guarantees it runs on every exit path, resources never leak even when the flow is wild.

3.7 finally — Guaranteed Cleanup

The demo prints both the failure message (`'ye to print hoga hi'`) and the `finally` message, proving `finally` executes on the error path too. That guarantee is what makes `finally` trustworthy:

- `return` inside `try` → `finally` still runs before the value is handed back.
- `raise` inside `except` → `finally` still runs, then the exception propagates.

- A `while True` loop may be abandoned by an exception → `finally` runs.

Use `finally` for anything that must always happen, and `with` for simpler auto-cleanup of file objects. Together they cover nearly every resource-management need in Python.

PYTHON

```
# finally
# else
try:
    f = open('sample1.txt', 'r')
except FileNotFoundError:
    print('file nai mili')
except Exception:
    print('kuch to lafda hai')
else:
    print(f.read())
finally:
    print('ye to print hoga hi')
```

4. Raising Exceptions Yourself

4.1 The raise Keyword

Errors do not have to arrive on their own — you can *throw* them deliberately with `raise`:

Why would you do that? Because detecting an invalid condition and failing fast is often better than continuing with broken state:

- **Input validation** — a negative price, an empty name, a wrong age.
- **Business rules** — not enough balance for a withdrawal.
- **Clarity** — raising a *specific* named exception makes the problem obvious to the caller.

`raise` accepts an exception class and an optional message; `str(e)` recovers the message on the catching side. A raised exception behaves exactly like a natural one: if it is inside a `try`, the matching `except` catches it; otherwise it propagates up the call chain. (Python fans of Java will recognise `raise` ≈ `throw` and `except` ≈ `catch`.) The one anti-pattern to avoid is raising a bare, generic `Exception` everywhere — name your errors precisely.

PYTHON

```
# raise Exception
# In Python programming, exceptions are raised when errors occur at runtime.
# We can also manually raise exceptions using the raise keyword.

# We can optionally pass values to the exception to clarify why that exception was raised
```

PYTHON

```
raise ZeroDivisionError('aise hi try kar raha hu')
```

Deep Dive : raise — manual exception

- `raise Exception('message')` — THROW an error yourself
- When: detect an invalid state and fail fast
- Custom business rules: not enough balance, wrong age
- Raise inside try → caught by except
- Common raises: `raise ValueError('invalid input')`
- raise with args → `str(e)` shows the message
- Can you raise a non-Exception? `BaseException` exists but it's an anti-pattern
- Re-raise: `raise` inside except (to propagate)
- The generic 'raise Exception' — prefer specific
- Java-style: checked vs unchecked — in Python everything is unchecked

PYTHON

```
raise ZeroDivisionError('aise hi try kar raha hu')
# Java
# try -> try
# except -> catch
# raise -> throw
```

PYTHON

```
raise ZeroDivisionError('aise hi try kar raha hu')
```

4.2 raise In Business Logic — The Bank Example

The next cell is the classic real-world use of `raise`: an object that refuses to enter an invalid state.

- `Bank.__init__(balance)` stores the starting balance.
- `withdraw(amount)` checks the amount *before* touching the balance:
 - `amount < 0` is nonsense → `raise Exception('amount cannot be -ve')`;
 - `self.balance < amount` is an overdraft → `raise Exception('paise nai hai tere paas')`;
 - otherwise the balance is safely reduced.
- The caller wraps the call in `try/except Exception as e` to catch the message, with an `else` clause that prints the new balance only on success.

Raising here is a form of *validation with a contract*: a method documents what it requires, checks it, and fails loudly rather than silently corrupting the account balance. That is far safer than a method that quietly returns a flag and leaves the data in an impossible state.

PYTHON

```

class Bank:

    def __init__(self,balance):
        self.balance = balance

    def withdraw(self,amount):
        if amount < 0:
            raise Exception('amount cannot be -ve')
        if self.balance < amount:
            raise Exception('paise nai hai tere paas')
        self.balance = self.balance - amount

obj = Bank(10000)
try:
    obj.withdraw(15000)
except Exception as e:
    print(e)
else:
    print(obj.balance)

```

5. Custom Exceptions

5.1 Creating Your Own Exception Classes

When the built-in list of exceptions is not expressive enough, define your own. A custom exception is just a class that subclasses `Exception`:

Why bother?

- **Domain vocabulary** — `BankError`, `AuthError`, `DatabaseDown` say more than a forest of generic `Exception`s.
- **Precise catching** — `except MyException` catches exactly that family, nothing else.
- **Cleaner libraries** — users can react to your specific failure types.

The `__init__` passes the message up to the base class so `str(e)` and `print(e)` work as usual. Because your class sits inside the standard hierarchy (`MyException` → `Exception` → `BaseException`), it can be caught either by name or by any ancestor. In the code below the messages are passed to the base this way; the next section shows a tiny slice-down variant where `__init__` prints the text directly.

PYTHON

```

class MyException(Exception):
    def __init__(self,message):
        print(message)

class Bank:

    def __init__(self,balance):
        self.balance = balance

    def withdraw(self,amount):
        if amount < 0:
            raise MyException('amount cannot be -ve')
        if self.balance < amount:
            raise MyException('paise nai hai tere paas')
        self.balance = self.balance - amount

obj = Bank(10000)
try:
    obj.withdraw(5000)
except MyException as e:
    pass
else:
    print(obj.balance)

```

PYTHON

```

class MyException(Exception):
    def __init__(self, message):
        super().__init__(message)

```

5.2 A Minimal Custom Exception

A leaner form prints the message straight from `__init__`:

Then `raise MyException('amount cannot be -ve')` prints its own message when raised, and the surrounding `try` / `except` can still catch it by name or just `pass`. In the demo, `withdraw(5000)` succeeds (the balance covers it), so no exception fires and the `else` branch prints the updated balance — 5000. This variant is great for compact examples, though storing `self.message` is more robust for code that must inspect the error later.

PYTHON

```

class MyException(Exception):
    def __init__(self, message):
        print(message)

```

5.3 Why Custom Exceptions Beat Generic Ones

A custom exception carries meaning in its *name*. Compare two handlers:

- `except Exception as e:` tells you almost nothing about *why* — any failure matches.

- `except InsufficientFunds:` tells you the business rule that failed, immediately.

That is why real libraries ship their own exceptions (`requests` raises `requests.exceptions.RequestException`, drivers raise session errors, etc.). When you define `MyException(Exception)`, you claim a place in the hierarchy: anything that catches `Exception` still catches you, but code that only wants *your specific* failures can catch exactly that one class. The comment-heavy cell below hints at this: create custom exceptions, and learn the built-in hierarchy, because the two together give you surgical control over error handling.

5.4 Real Scenario — Domain Errors In Action

The `Google` / `SecurityError` demo at the end of the session packs everything together:

- `SecurityError(Exception)` carries a message and an extra `logout()` method — custom behaviour travels with the error.
- `Google.login(email, password, device)` raises `SecurityError` when the login device does not match the registered one — a security-domain violation.
- The caller wraps the login in `try/except SecurityError` and, on failure, does the right thing: logs the user out via `e.logout()`.
- `else` prints the user's name only when login succeeded; `finally` reports that the database connection has been closed.

Read the final output carefully: the security message prints, then `logout`, then the `finally` line — the exception path, the domain response and the guaranteed cleanup, all working together.

5.5 Exceptions As Data Integrity Guards

Think of `raise` inside business methods as a defensive wall:

- `withdraw()` validates before mutating — a guard that keeps the balance non-negative.
- A login loop checks the device and aborts with `SecurityError` rather than granting access.
- Methods document their raising conditions, so callers know which `except` clause to prepare.

Compared with returning error codes, exceptions are *explicit*: they cannot be silently ignored, they carry a message, and they integrate with the class hierarchy. Combined with OOP, this style produces clean code where data integrity is enforced at the source instead of being checked four call-sites away. Log a meaningful message when you raise — it doubles as a debugging aid when tracking down *why* a rule fired.

6. The Exception Hierarchy

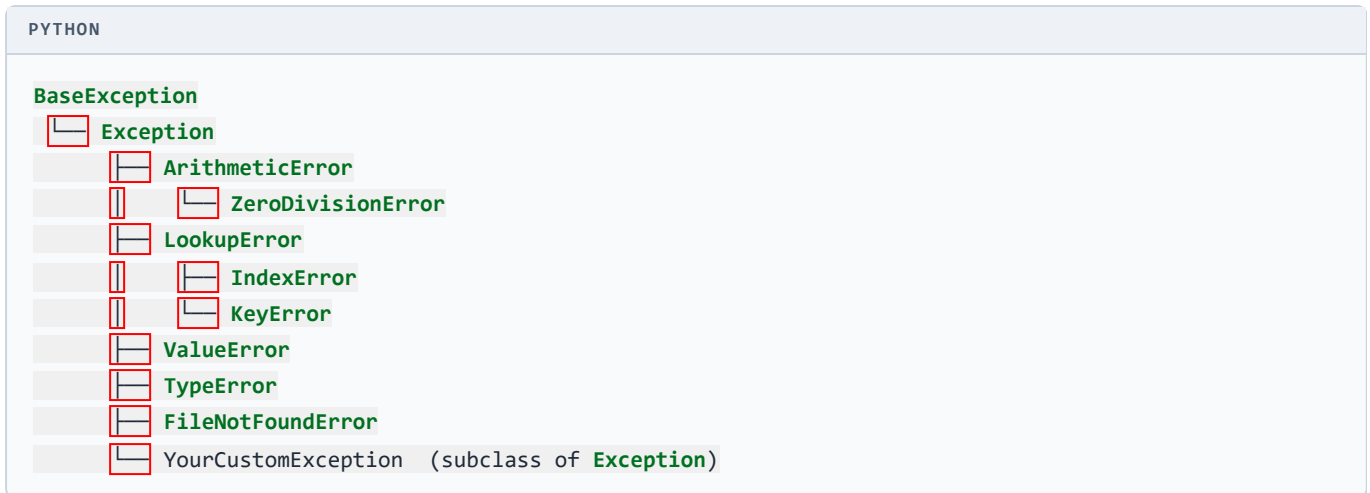
6.1 One Tree That Explains Everything

All exceptions hang from a single tree:

What the tree tells you:

- `BaseException` is the root; `Exception` is the parent of everything you will normally raise or catch.
- Catching a **parent** catches its **children**: `except LookupError` handles both `IndexError` and `KeyError`.

- `KeyboardInterrupt` and `SystemExit` are *siblings* of `Exception` under `BaseException` — a bare `except Exception` deliberately does **not** catch them.
- Your custom classes plug into the tree where you put them, inheriting all ancestor behaviour.



6.2 Navigating The Hierarchy

Practical takeaways:

- **Order matters:** put child exceptions first, generic parents last. `except (IndexError, KeyError):` groups siblings with a tuple.
- **Inspect with MRO:** `MyException.__mro__` shows your class's full ancestor chain.
- **Catch-all meaning:** `except Exception:` covers user-program errors but not `KeyboardInterrupt` or `SystemExit` — exactly the safety you usually want in an app's outer shell.
- **Docs:** `help('builtins')` or `dir(__builtins__)` lists the complete built-in family when you need the full picture.

Knowing the parents is knowing what your `except` catches — and that single fact prevents the most embarrassing handling bugs: catching the wrong sibling, forgetting a child, or ducking the keyboard interrupt the user expects to stop the program.

PYTHON

```

class SecurityError(Exception):

    def __init__(self,message):
        print(message)

    def logout(self):
        print('logout')

class Google:

    def __init__(self,name,email,password,device):
        self.name = name
        self.email = email
        self.password = password
        self.device = device

    def login(self,email,password,device):
        if device != self.device:
            raise SecurityError('bhai teri to lag gayi')
        if email == self.email and password == self.password:
            print('welcome')
        else:
            print('login error')

obj = Google('nitish','nitish@gmail.com','1234','android')

# ...
else:
    print(obj.name)
finally:
    print('database connection closed')

```

Namespaces, Scope & Decorators

1. Namespaces and the LEGB Rule

1.1 A namespace is a dictionary of names

A **namespace** is exactly what it sounds like: a space that holds names. Programmatically, it is a dictionary whose keys are identifiers (like `a`, `temp`, `print`) and whose values are the objects they refer to. Python maintains four of them around every piece of code:

NAMESPACE	WHAT IT HOLDS
Built-in	<code>print</code> <code>len</code> <code>dict</code> <code>max</code> ...
Global	everything defined at the top of a module
Enclosing	variables of outer functions around a nested function
Local	names created inside the currently executing function

A **scope** is the textual region of the program where a given namespace is directly accessible — where a name can be used without a qualifier. The two concepts go together: the namespace is the dictionary, the scope is the territory.

When Python meets a name in your code, it does not guess — it searches from the inside out, strictly in this order: **Local**, **Enclosing**, **Global**, **Built-in**. The first hit wins. If no scope in the chain contains the name, Python raises `NameError`. Memorise the acronym LEGB; it is tested in interviews and it silently explains most of the surprising behaviour you will see in the code cells that follow — for instance, why a variable named `a` inside a function and a variable named `a` at the top level are two completely different objects living in different namespaces.

Local vs Global scope

- **Global** — file top-level, visible everywhere
- **Local** — created inside a function, accessible only there
- `b` created inside `temp()` won't be found outside
- Namespace = a dictionary of names, scope = a region

```

PYTHON

a = 2                # global

def temp():
    b = 3            # local - sirf is function ke andar
    print(b)

temp()
print(a)             # global accessible
# print(b)           # NameError! `b` Local hi tha

```

Deep Dive : Namespace & local/global scope

- **Namespace** — a mapping of names (variables, functions) to objects
- Every function has its OWN namespace
- Module namespace, function namespace, builtin namespace
- Global: module level, accessible everywhere
- Local: inside a function, not visible outside the function
- Scope = the region where a name is visible

- LEGB lookup order: Local → Enclosing → Global → Builtin
- Strategy: read globals freely inside; writing needs the keyword
- Namespaces are daisy-chained (lexically)

PYTHON

```
# Local and global
# global var
a = 2

def temp():
    # local var
    b = 3
    print(b)

temp()
print(a)
```

PYTHON

```
a = 2                # global var – module namespace

def temp():
    b = 5            # local var – function namespace
    print(b)
temp()
print(a)             # ✓ global outside
# print(b)           # NameError – b local scope me
```

Same name local vs global

- An assignment inside a function creates a **local**
- The global `a` stays as it is — this is shadowing
- The two namespaces are separate; `a` is its own thing in each place

PYTHON

```
a = 2                # global

def temp():
    a = 3            # LOCAL nayi `a` – global ko nahi chhoTa
    print(a)

temp()              # 3
print(a)            # 2 – global untouched
```

Deep Dive : Same-name shadowing

- An assignment `a = 5` in a function → a **new local** variable
- Without the assignment, the global `a` is only READ
- An assignment anywhere in the function → the whole function treats it as local (shadowing)

- The same name is allowed — the scope differentiates
- Shadowing hides the global inside the function scope
- To prevent this: you need `global a`
- No local + global exists → reads the global (pass-through)
- This is called "LEGB": the function scope comes first
- Beginners get confused here — take 10 minutes to conceptualize it

PYTHON

```
# Local and global -> same name
a = 2

def temp():
    # Local var
    a = 3
    print(b)

temp()
print(a)
```

PYTHON

```
# Local and global -> Local does not have but global has
a = 2

def temp():
    # Local var
    print(a)

temp()
print(a)
```

Editing a global — the global keyword

- Without declaring it, `a += 1` gives `UnboundLocalError` (reading is also local)
- `global a` — tell the function "use this global one"
- Reading a global (without `global`) is allowed, modifying is not
- Use `global` sparingly — to avoid bad practice

PYTHON

```

a = 2

def temp():
    global a    # ab mai global `a` ko badluga
    a += 1
    print(a)

temp()         # 3
print(a)       # 3 - global change ho gaya

```

Deep Dive : Editing globals — the global keyword

- Without `global` : `a += 1` inside → `UnboundLocalError`
- `global a` — tells the function "use this global"
- When needed: counters, config, shared state updates
- Risky — it hurts readability (hidden dependency)
- Modern code: prefer return values / class-state design
- Common error: `UnboundLocalError: local variable referenced before assignment`
- Calling a method on a global list or dict (append) — no global needed (mutation is fine)
- Rebinding (`a = x`) needs global; mutating (`L.append`) does not
- global must be at the top of the function

PYTHON

```

# Local and global -> editing global
a = 2

def temp():
    # Local var
    a += 1
    print(a)

temp()
print(a)

```

PYTHON

```

a = 2

def temp():
    # Local var
    global a
    a += 1
    print(a)

temp()
print(a)

```

Creating a global from a local

- With `global a`, an assignment creates a new entry in the global namespace
- You can create a global from inside a function too
- Rarely needed — usually compared against module level

PYTHON

```
def temp():
    global a
    a = 1      # global me naya name ban gaya!
    print(a)

temp()
print(a)      # 1 - ab global me hai
```

Deep Dive : Creating a global from a function

- `global c` + assignment → CREATE in the global namespace
- Making a variable global from within a function = sloppy design
- Prefer returning and assigning at the module level:
- Global-from-inside → maintenance pain
- A shared state's side effects become unclear
- The global registry is an anti-pattern (global styling at its worst)
- A rare real need — module-level config via constants is fine
- Encapsulation via classes with attributes
- namespaces live as long as the module imports are alive

PYTHON

```
# Local and global -> global created inside local
def temp():
    # Local var
    global a
    a = 1
    print(a)

temp()
print(a)
```

PYTHON

```
def temp():
    global c
    c = 100      # global namespace me naya banaya
temp()
print(c)        # 100 - global bana
```

Parameters are local

- Function parameters belong to the **local scope** too
- After the call, `z` vanishes outside
- This is why every function handles its own parameters

PYTHON

```
def temp(z):
    print(z)      # z function-ka-Local hai

a = 5
temp(5)          # z = 5
# print(z)       # NameError! z sirf function ke andar
```

Deep Dive : Parameters are local

- Parameters = the function's local namespace
- Values are passed copy-in (for immutables)
- Outside the scope `z` is undefined
- Every call creates a fresh local frame
- A mutable param -> the reference is shared (mutation affects the caller)
- Default param trap: `def f(L=[])` — shared default! use `None`
- Function local variables vanish after return (except closures)
- Call arguments are evaluated at call time
- Scope rule: parameters behave like locals

PYTHON

```
# Local and global -> function parameter is Local
def temp(z):
    # Local var
    print(z)

a = 5
temp(5)
print(a)
print(z)
```

PYTHON

```
def temp(z):      # z is LOCAL to temp
    print(z)
temp(10)          # prints 10
z                 # NameError outside
```

2. Built-in Scope

2.1 Names that are always available

The **built-in namespace** is the outermost layer of the LEGB chain — the fallback when Local, Enclosing, and Global all come up empty. It contains every built-in Python function and constant: `print`, `len`, `type`, `max`, `min`, `list`, `int`, `input`, and hundreds more. The interpreter populates it at startup, which is why `print` works in every function with no import at all.

The notebook shows two things about it. First, we can inspect the whole built-in namespace directly:

`dir()` returns every name in the module, so you can browse the full built-in catalogue. Second, the namespace is only a *fallback*, not a fortress — we can shadow a built-in by defining our own name in an inner scope:

After that, `max` resolves to our function (Local/Global beats Built-in), which usually breaks code expecting the real `max`. The lesson: treat built-in names as reserved — there is almost never a good reason to redefine `max`, `list`, or `len`. The lookup order also guarantees that, unless we shadow it, a built-in is reachable from anywhere in the program.

PYTHON

```
# built-in scope
import builtins
print(dir(builtins))
```

PYTHON

```
import builtins
print(dir(builtins))
```

Built-in scope

- **Built-in** = the outermost scope — `print`, `len`, `type` ...
- With `dir(builtins)` you can see all the built-in names
- Built-ins are always available from inside a function
- If you override one, you shadow it — see the next cell

PYTHON

```
# renaming built-ins
L = [1,2,3]
print(max(L))
def max():
    print('hello')

print(max(L))
```

PYTHON

```
import builtins
print(dir(builtins)) # sab built-in names ki list
```

3. Enclosing Scope and nonlocal

3.1 Nested functions read the outer scope

So far we examined Local and Global. The **Enclosing** scope sits between them and only exists when a function is defined *inside* another function:

When `inner` looks for `a`, its own Local namespace has no `a`, so the search moves to the Enclosing scope — `outer`'s locals — and finds it. This is a **closure** in action: the inner function carries a reference to its enclosing environment, even after `outer` returns. Closures are the foundation of decorators.

Reading an enclosing variable is free, but *reassigning* one is not. Because `a += 1` inside `inner` counts as an assignment, Python would treat `a` as local to `inner` and complain that it was used before assignment. The `nonlocal` keyword fixes this — it tells the function "the `a` I mean belongs to the enclosing scope":

Compare the two keywords: `global` points at the module-level namespace, `nonlocal` points at the nearest enclosing *function* namespace. When a closure needs to keep running state, `nonlocal` is how it mutates that state.

PYTHON

```
def outer():
    a = 1
    def inner():
        print(a) # finds `a` in the enclosing scope
    inner()
    print('outer function')

outer()
```

PYTHON

```
def outer():
    a = 1
    def inner():
        nonlocal a
        a += 1
    inner()
    print(a) # 2
```

Deep Dive : Enclosing / nonlocal scope

- **Enclosing scope** — the outer function's local within a nested function
- inner reads the outer's variables (a closure)
- LEGB: inner first its own local, then enclosing (outer), then global

- To WRITE the outer's variable → you need `nonlocal`
- `nonlocal` vs `global` — `nonlocal` = the enclosing function
- Closures capture the enclosing environment
- Loop/closure bug: late binding (functions capture the variable, not the value)
- Foundation for decorators! (the wrapper reads the function)
- Interview: closures + `nonlocal`

PYTHON

```
# Enclosing scope
def outer():
    def inner():
        print(a)
    inner()
    print('outer function')

outer()
print('main program')
```

PYTHON

```
def outer():
    a = 10 # enclosing scope (outer ka local)
    def inner():
        print(a) # closure - outer ka a read
    inner()
```

The `nonlocal` keyword

- `nonlocal` — to change an enclosing (outer) variable
- Reading is allowed; declare `nonlocal` to modify
- This is core to closures (decorators are built on it)

PYTHON

```
# nonlocal keyword
def outer():
    a = 1
    def inner():
        nonlocal a
        a += 1
        print('inner', a)
    inner()
    print('outer', a)

outer()
print('main program')
```

PYTHON

```
def outer():
    a = 1
    def inner():
        nonlocal a # outer ka `a` modify
        a += 1
        print('inner', a)
    inner()

outer() # inner 2
```

4. Decorators

4.1 Functions that decorate functions

A **decorator** is a function that receives another function as input, adds some functionality (the "decoration"), and returns the enhanced version. It is possible only because Python functions are **first-class citizens** — they are ordinary objects that can be passed around like any value:

Notice there is no `()` after `square` in `modify(square, 4)` — we hand over the function object, and `modify` calls it through its parameter. Any function that takes or returns another function is a **higher-order function**, and a decorator is exactly that: a higher-order function applied at definition time.

The first decorator in the notebook wraps its target with banner lines:

`@my_decorator` is pure sugar for `hello = my_decorator(hello)`. From then on, the name `hello` refers to `wrapper`, which runs the original body in between its own statements. The wrapper is a *closure* — it captured `func` — and that captured reference is what lets the decoration "wrap" the original.

Python ships a small set of built-in decorators (`@staticmethod`, `@classmethod`, `@property`, `@abstractmethod`), but users can write their own for logging, timing, validation, and caching.

PYTHON

```
def modify(func, num):
    return func(num)

def square(num):
    return num ** 2

modify(square, 4) # 16 - we passed the function itself
```

PYTHON

```
def my_decorator(func):
    def wrapper():
        print('*****')
        func()
        print('*****')
    return wrapper

@my_decorator
def hello():
    print('hello')
```

Functions are first-class citizens

- In Python functions are also values — pass, return, store, all fine
- `func` as an argument — a higher-order function
- Decorators exist because of this power

PYTHON

```
def modify(func, num):
    return func(num)

def square(num):
    return num ** 2

modify(square, 4)    # 16 — function as value pass
```

Deep Dive : Functions are first-class

- Functions are objects: pass as args, store, return
- Already seen in Session 6 — recap: a `func` reference is passed
- HOF: a function that takes/returns functions
- `modify(square, 3)`: `square` → `modify`'s parameter
- Decorators USE this — function in, function out
- `map/filter/reduce` are built on it
- Store: `L = [square, cube]`
- `identity fn == fn`
- This sets the stage — decorators = a function wrapping a function

PYTHON

```
# Python are 1st class function
```

```
def modify(func,num):
    return func(num)
```

```
def square(num):
    return num**2
```

```
modify(square,2)
```

PYTHON

```
def modify(func, num):
    return func(num)
```

```
def square(x):
    return x**2
```

```
result = modify(square, 3)    # function passed as arg
print(result)                # 9
```

First decorator

- A decorator = a function that takes a function and returns a wrapper
- `@my_decorator == hello = my_decorator(hello)`
- When you call `hello()`, the wrapper runs — around the original
- Pre/post processing — logging, timing, checks

PYTHON

```
def my_decorator(func):
    def wrapper():
        print('*****')
        func()
        print('*****')
    return wrapper
```

```
@my_decorator
def hello():
    print('hello')
```

Deep Dive : First decorator — closure + wrapper

- `@decorator` — syntactic sugar = `func = decorator(func)`
- The decorator receives `func`, defines `wrapper`, returns wrapper
- wrapper extends the behavior (before/after)
- Closure: the wrapper holds the func reference
- Calling `hello()` → the actual wrapper runs → `func()` runs

- Return value passthrough: `result = func(...)` + `return result`
- `**args/**kwargs` forwarding — preserves the signature
- Real usage: logging, timing, auth, caching
- Decorators = HOFs applied at definition time
- Behavior injection without touching the original

PYTHON

```
# simple example

def my_decorator(func):
    def wrapper():
        print('*****')
        func()
        print('*****')
    return wrapper

def hello():
    print('hello')

def display():
    print('hello nitish')

a = my_decorator(hello)
a()

b = my_decorator(display)
b()
```

PYTHON

```
# Better syntax?
# simple example

def my_decorator(func):
    def wrapper():
        print('*****')
        func()
        print('*****')
    return wrapper

@my_decorator
def hello():
    print('hello')

hello()
```

Real decorator — timer

- `*args` / `**kwargs` — pass every kind of argument through the wrapper
- Timing, logging, access checks — what decorators are perfect for

- Original function unchanged, yet the call is enriched

PYTHON

```
import time

def timer(func):
    def wrapper(*args):
        start = time.time()
        func(*args)
        end = time.time()
        print('took', end - start)
    return wrapper

@timer
def do_something():
    time.sleep(2)
```

Deep Dive : Real decorator — timer

- The decorator's real power: cross-cutting concerns
- A timer measures any function without editing its code
- `func.__name__` — introspection is preserved (mostly)
- siMetrics: time, log, retry, cache
- Code simplification: wrap + annotation
- A decorator is a HOF returning a wrapper
- Function composition with `@a @b` — `a(b(f))`
- The "extra capability" is readable right at the definition
- Frameworks (Flask routes, Django ORM) use them heavily

PYTHON

```
# anything meaningful?
import time

def timer(func):
    def wrapper(*args):
        start = time.time()
        func(*args)
        print('time taken by', func.__name__, time.time()-start, 'secs')
    return wrapper

@timer
def hello():
    print('hello wolrd')
    time.sleep(2)

@timer
def square(num):
    time.sleep(1)
    print(num**2)

@timer
def power(a,b):
    print(a**b)

hello()
square(2)
power(2,3)
```

PYTHON

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'func. {func.__name__} took {end-start:.4f}s')
    return result
    return wrapper

@timer
def slow_task():
    time.sleep(1)
slow_task() # slow_task took 1.0004s
```

5. Wrapping Without Losing Identity

5.1 The metadata problem and functools.wraps

There is a subtle bug hiding inside every plain decorator. When `@my_decorator` reassigns `hello = my_decorator(hello)`, the object now bound to `hello` is the `wrapper` function — and the wrapper carries its **own** metadata, not the original's:

The name, docstring, and even the signature that debugging tools, `help()`, and documentation generators rely on have been replaced by the wrapper's. The function behaves correctly but *identifies* incorrectly.

The fix lives in the standard library:

`@wraps(func)` copies `__name__`, `__doc__`, `__module__`, and the stored signature from the original function onto the wrapper, so an outer decorator no longer masquerades as a stranger. This is the universal professional convention — a decorator without `@wraps` breaks introspection, which is worse than it sounds: tracebacks, `help()`, and IDE tooltips all report the wrapper instead of the wrapped function.

PYTHON

```
@my_decorator
def greet():
    '''Print a greeting.'''
    print('hi')

greet.__name__ # 'wrapper' – the original name lost!
greet.__doc__  # None – the docstring lost!
```

PYTHON

```
from functools import wraps

def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        ...
    return wrapper
```

Deep Dive : Problem with a plain wrapper

- A decorated function loses its metadata: `__name__`, `__doc__`, `__module__`
- Introspection tools see the wrapper, not the original
- Fix: `functools.wraps`:
- `@wraps(func)` copies **name**, **doc**, signature-ish
- Best practice: ALWAYS use `@wraps`
- debug & trace stay clean
- othera — `functools.wraps` is essential for professional decorators
- Without it — the API/documentation is broken

PYTHON

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        ...
    return wrapper

@my_decorator
def greet():
    '''docstring'''
    ...

print(greet.__name__) # ❌ 'wrapper' - name lost!
print(greet.__doc__) # ❌ None - doc lost!
```

PYTHON

```
from functools import wraps
def my_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        ...
    return wrapper
```

6. Decorators That Take Arguments

6.1 The three-layer onion

So far our decorators took only the function. What if the decoration should depend on a *parameter* — like a type we want to validate against? Look at the syntax the notebook uses:

Note the extra parentheses: `@name(args)` means `name(args)` runs *first*, returning a function, and *that* function receives `square`. We therefore need **three** nested layers instead of two:

Layer 1 captures `data_type`, layer 2 captures `func`, layer 3 performs the runtime check and calls the original. The full evaluation is `square = sanity_check(int)(square)`.

This gives us *parameterised* decorations: type validation, role-based access control, rate limits. Flask's `@app.route('/path')` is precisely this pattern. The three layers look intimidating, but the mental model is an onion — each layer closes over the previous one's variables: arguments, then function, then the check-and-call wrapper.

PYTHON

```
@sanity_check(int)
def square(num):
    print(num ** 2)
```

PYTHON

```
def sanity_check(data_type):           # Layer 1: takes OUR argument
    def outer_wrapper(func):          # Layer 2: takes the target function
        def inner_wrapper(*args):    # Layer 3: the real wrapper logic
            if type(*args) == data_type:
                func(*args)
            else:
                raise TypeError('datatype check failed')
        return inner_wrapper
    return outer_wrapper
```

Deep Dive : Decorators with arguments

Structure: 3 layers!

- `@deco(arg)` = call deco first → returns the actual decorator
- Layer 1: takes OUR argument (data_type)
- Layer 2: takes the target function
- Layer 3: the wrapper does the runtime work — the arg check
- Usage: type validation, auth roles, rate limits
- Real parameterized decorators everywhere (flask route!)
- The extra () pair = the factory call
- Execution: `sanity_check(int)(square)`
- Powerful but the 3 layers are confusing — draw the onion diagram

PYTHON

```
@checkdt(int)
def square(num):
    print(num**2)
```

PYTHON

```
@checkdt(int)           # decorator that TAKES args
def square(num):
    print(num ** 2)
```

The structure of sanity_check

- 3 layers = args + func + the args call
- `@name(args)` — the first call is with the arguments
- Then you get the decorator function — and the original function goes onto it
- Remember the layer counting: args → func → wrapper logic

PYTHON

```
def sanity_check(data_type):
    def outer_wrapper(func):
        def inner_wrapper(*args):
            if type(*args) == data_type:
                func(*args)
            else:
                raise TypeError('Ye datatype nai chalega')
        return inner_wrapper
    return outer_wrapper

@sanity_check(int)
def square(num):
    print(num**2)

@sanity_check(str)
def greet(name):
    print('hello',name)

square(2)
```

PYTHON

```
def sanity_check(data_type):          # Layer 1: args
    def outer_wrapper(func):         # Layer 2: function
        def inner_wrapper(*args):    # Layer 3: real Logic
            if type(*args) == data_type:
                print('number is integer')
            else:
                print('number is not integer')
        return inner_wrapper
    return outer_wrapper

@checkdt(int) # == square = sanity_check(int)(square)
```

Edge Cases and Boundaries

6.2 DEEP DIVE — MUTATION ACROSS THE CALL BOUNDARY

- Objects are passed **by reference** into functions.
- Mutation inside → the caller's object is affected: same object, shared memory.
- Immutables (int/str): rebinding inside the function is invisible outside.
- Mutables (list/dict/object): in-place changes are visible outside.
- This causes classic bugs where a function silently rewrites caller data.
- Defensive approach: copy the object inside the function first.
- Mental model: variables are arrows, objects are boxes; function parameters copy the arrow, not the box.

The cells below show it live: `greet(p)` receives the same id as `p`, sets `person.name = 'ankit'`, and `print(p.name)` prints `ankit` — proof the change happened on the original object.

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- class design, dunder methods and inheritance
- file I/O and context managers
- exception hierarchies and error strategy
- decorators and generators

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

1.2 DEEP DIVE — EVERYTHING IS AN OBJECT

```
L = [1,2,3]
```

```
L.upper()
```

PYTHON

1.2 DEEP DIVE — EVERYTHING IS AN OBJECT

```
s = 'hello'
s.append('x')
```

PYTHON

1.2 DEEP DIVE — EVERYTHING IS AN OBJECT

```
L = [1,2,3]
print(type(L))
```

PYTHON

1.2 DEEP DIVE — EVERYTHING IS AN OBJECT

```
L = [1, 2, 3]
L.upper()      # AttributeError – 'list' object has no attribute 'upper'
s = 'hello'
s.append('x')  # AttributeError – 'str' object has no attribute 'append'
```

PYTHON

2.2 DEEP DIVE — OBJECT: THE INSTANCE

```
# syntax to create an object

#objectname = classname()
```

PYTHON

2.2 DEEP DIVE — OBJECT: THE INSTANCE

```
# object literal
L = [1,2,3]
```

PYTHON

3. CONSTRUCTORS & LITERALS

```
L = [1, 2, 3]    # object literal (shortcut)
L = list()       # constructor call (zero args)
s = str()        # constructor → ''
```

PYTHON

CONSTRUCTORS & LITERALS

```
L = [1, 2, 3]    # object literal (shortcut)
L = list()       # constructor call (zero args)
s = str()        # empty string
```

Glossary

method

called with a dot inside the type (``L`.

PascalCase

every word capitalized, no spaces — ``HelloWorld``.

snake_case

lowercase words joined by underscores — ``my_var``, ``my_func``.

Reference variables

an object lives in memory; a variable only **points** at it, and many variables can point at the same object.

Pass by reference

when you hand an object to a function, the function mutates the very same object — changes are visible outside.

Encapsulation

hiding an object's data behind methods with the ``__`` prefix so outside code cannot touch it directly.

Static members

class-level variables and ``@staticmethod`` methods shared by all instances, usable even without an object.

Static/class variable

class-level, one copy shared by everyone.

Instance variable

each object's own separate copy.

Aggregation

embed an object inside another (``Customer`` has an ``Address``).

Inheritance basics

``class Child(Parent):``, what gets inherited, and private members that stay private.

Overriding

the child replaces a parent method with its own version.

Inheritance types

single parent chains down to the diamond problem of multiple inheritance, resolved by Python's MRO.

Polymorphism

overriding, overloading (via default arguments) and operator overloading — the same `+` means different things on different types.

Abstraction

an `ABC` with `@abstractmethod` that *forces* children to implement certain methods.

Is-A

a Student IS-A User.

Constructor

because `Student` defines its own `\_\_init\_\_`, calling `Student()` runs only the child's — `rollno` is set, but `name`/`gender` are NOT, unless the child calls `super().\_\_init\_\_`.

Methods

`login()` was inherited, and the child wrote `enroll()` — `s.login()` and `s.enroll()` both work.

Snapshot honesty

the cell's saved output shows `nitish` because it was captured when `Student` had NO `\_\_init\_\_` of its own. Run the current source fresh and `print(s.name)` would raise `AttributeError` — a live reminder that *the constructor decides which attributes exist*.

Single

the everyday case: one base, one child.

Overloading

Python has no real overloading — the last definition wins — so emulate it with default arguments (`area(2)` vs `area(3,4)`).

Operator overloading

you met `\_\_add\_\_` in Session 7; `+` behaves per type.

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

☐

1. Function vs Method

☐

1.2 Deep dive — everything is an object

☐

2. Object Syntax

☐

Object syntax

☐

2.2 Deep dive — object: the instance

☐

3. Constructors & Literals

☐

Constructors & literals

☐

3.2 Deep dive — constructors vs literals

☐

4. Naming Conventions

☐

Naming conventions

☐

4.2 Deep dive — naming conventions

☐

5. Making a Class + Constructor

☐

Making a class + constructor

☐

5.2 Deep dive — the constructor auto-runs

☐

5.3 Deep dive — building a real class with `__init__`

☐

6. Creating Objects & Identity

☐

Creating objects + identity

☐ 6.2 Deep dive — identity, not just equality

☐ 7. Function vs Method — `len` vs `append`

☐ len vs append

☐ 7.2 Deep dive — generic function vs type-specific method

☐ 8. The Constructor Runs By Itself